A

Report On

Internship

(Under- "Industry Internship &Project" Track)

At

Messen Labs.LLP, Bengaluru

Submitted

in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Mechanical Engineering

by

Mr. Padmanabh Dnyanesh Kulkarni. (2256034)

Under the Guidance of

Prof. G. L. Suryawanshi

Mechanical Engineering Department

K. E. Society's

Rajarambapu Institute of Technology, Rajaramnagar

(An autonomous Institute, Affiliated to Shivaji University)

2024-2025

K. E. Society's

Rajarambapu Institute of Technology, Rajaramnagar

(An autonomous Institute, Affiliated to Shivaji University)

Department of Mechanical Engineering

CERTIFICATE

This is to certify that the project under Industry Internship & Project (IIP) track completed at " Messen Labs"is the bona fide work submitted by the following student, to the Rajarambapu Institute of Technology, Rajaramnagar during the academic year 2024-25, in partial fulfillment for the award of the degree of B. Tech in Mechanical Engineering under our supervision. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree.

Name of Students	Roll Number
1. Padmanabh Kulkarni	2256034

Date:

Place : Rajaramnagar

Prof. G. L. Suryawanshi
Industry Internship & Project
Mentor(College)

Mr. Nikhilesh Prabhu
Industry Internship & Project
Mentor(Industry)

External Examiner

Prof. R. A. Magdum
Training &Placement Coordinator

Prof. S. B. Kumbhar
Head of Department

DECLARATION

I declare that this report reflects my thoughts about the subject in my own words. I have sufficiently cited and referenced the original sources, referred or considered in this work. I have not plagiarized or submitted the same work for the award of any other degree. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute.

Sr. No.	Student Name	Roll No	Signature
1.	Padmanabh Kulkarni	2256034	

Date:

Place : RIT, Rajaramnagar.

ACKNOWLEDGEMENT

I take this opportunity to thank all those who have contributed in the successful completion of a Project Under Industry Internship & Project (IIP) track at Messen Labs. I sincerely wish to express my gratitude to Industry Internship & Project (IIP) Mentor Prof. G. L. Suryawanshi for full support, expert guidance, and encouragement and kind cooperation throughout the Internship work. I am greatly indebted to him for his help throughout project work. I express my sincere gratitude towards Dr. S. B. Kumbhar, Head of the Department, Mechanical Engineering, for providing necessary facilities, guidance and support.

I respect and thank Mr. Nikhilesh Prabhu(Industry Internship & Project (Industry Mentor) for providing me an opportunity to do an Internship in Messen Labs and giving us all support and guidance which made me complete the internship duly. I am extremely thankful to him for providing such a nice support and guidance, although he had busy schedule managing the corporate affairs.

I thank Prof. R. A. Magdum (Dept TPC) for providing Internship & Project Opportunity in an Industry.

I am thankful to and fortunate enough to get constant encouragement, support and guidance from all Teaching staffs of Mechanical Engineering Department, which helped me in successfully completing internship.

Nevertheless, I express my gratitude toward my family members for their kind co-operation and encouragement which helped me in the completion of this internship.

ABSTRACT

This study addresses the challenge of efficiently analysing large datasets by comparing automated tools (RapidMiner and Altair AI Studio) with traditional manual coding methods. As data volumes grow, manual analysis becomes time-consuming and error-prone, necessitating the adoption of automated solutions. The research evaluates the effectiveness, accuracy, and efficiency of these approaches in processing structured and unstructured data.

A comparative methodology was employed, applying both automated tools and manual coding to identical datasets. Key metrics included processing time, accuracy, scalability, and user effort. RapidMiner or Altair AI Studio demonstrated significant advantages in speed and repeatability, reducing analysis time compared to manual methods. However, manual coding provided deeper contextual insights for complex, nuanced data.

The findings suggest that while automated tools excel in handling large-scale, repetitive tasks, manual analysis remains valuable for interpretative tasks. A hybrid approach leveraging both methods is recommended for optimal results. This study contributes to the ongoing discourse on data analysis efficiency, offering practical insights for researchers and practitioners.

CONTENTS

Chapter 1 : Introduction

1.1 Background and Problem Statement

In the era of big data, organizations and researchers face the challenge of efficiently processing and analysing vast amounts of structured and unstructured data. Traditional manual coding methods, while precise in some cases, are time-consuming, labour-intensive, and prone to human error. On the other hand, automated data analysis tools such as RapidMiner promise faster processing, scalability, and reduced manual effort. However, concerns remain about their accuracy, flexibility, and ability to handle complex, context-dependent datasets.This study addresses the following key questions:

Understanding these trade-offs is crucial for businesses, researchers, and data analysts seeking to optimize their workflows while maintaining high-quality insights.

1.2 Importance of the Study

As data volumes continue to grow exponentially, the demand for efficient analysis methods has never been higher. While previous studies have examined individual tools or manual techniques, there is limited comparative research evaluating RapidMiner against traditional manual coding.

This study contributes to the field by:

- Providing a systematic comparison of automated vs. manual data analysis methods.
- Evaluating real-world applicability across different data types (structured vs. unstructured).
- Proposing a hybrid methodology that leverages the strengths of both approaches.
- Offering practical recommendations for organizations choosing between automation and manual analysis.
- . The findings will help data professionals make informed decisions about tool selection, workflow optimization, and resource allocation.

1.3 Objectives of the Study

The primary objectives of this research are:

1. To compare the performance of RapidMiner and Altair AI Studio with manual coding in terms of:
    - Processing speed
    - Accuracy and error rates
    - Scalability for large datasets
    - Ease of implementation and user effort

Introduction

1. To identify scenarios where automated tools outperform manual methods and vice versa.
2. To propose a hybrid framework that integrates the best aspects of both approaches.

1.4 Scope and Limitations

Scope

- The study focuses on text and tabular data analysis, covering both structured (e.g., CSV, Excel) and unstructured (e.g., natural language text) datasets.
- The comparison includes preprocessing, classification, and sentiment analysis tasks.
- The evaluation metrics include time efficiency, accuracy, and interpretability.

Limitations

- The study does not cover real-time streaming data or highly specialized domains (e.g., genomic data analysis).
- The performance of automated tools may vary based on computational resources (CPU, RAM).
- Manual coding results depend on human expertise, introducing subjectivity in some cases.

1.3 Objectives of the Study

This research aims to provide a comprehensive evaluation of automated data analysis tools (RapidMiner and Altair AI Studio) versus manual coding methods. The specific objectives are:

1. **Comparative Performance Analysis**

Processing Speed: Measure the time taken by each method to complete identical tasks (e.g., data cleaning, feature extraction, model training).

- Automated tools are expected to process large datasets faster, but manual coding may offer finer control in iterative refinement.
- **Accuracy and Error Rates:** Assess discrepancies in results (e.g., misclassifications in sentiment analysis, missing values in preprocessing) to determine which approach yields more reliable outcomes.

- **Scalability:** Evaluate how each method handles increasing data volumes (e.g., 10k vs. 1M records). Automated tools should theoretically scale better, but manual coding might still excel in small, complex datasets requiring human judgment.
- **Ease of Implementation and User Effort:** Compare setup complexity (e.g., tool configuration, scripting) and ongoing maintenance (e.g., debugging, updates). Manual coding demands domain expertise, while automation requires technical proficiency in tool-specific workflows.

2. Scenario-Based Identification
   - Identify use cases where automation excels (e.g., repetitive tasks like log parsing, large-scale preprocessing).
   - Highlight contexts where manual coding remains superior (e.g., ambiguous text interpretation, ethical bias detection).
3. Hybrid Framework Proposal
   - Develop a decision model to guide analysts on when to use automation, manual methods, or a combination (e.g., automate preprocessing + manual validation).

1.4 Scope and Limitations

Scope

1. Data Types:
   - Structured Data (CSV, Excel): Focus on numerical/categorical analysis (e.g., sales trends, customer segmentation).

Unstructured Data (Natural Language Text): Evaluate sentiment analysis, topic modeling, and text classification

2. Tasks Evaluated:
   - Preprocessing: Handling missing values, normalization, and feature engineering.
   - Classification: Supervised learning (e.g., spam detection) using both tools and manual rule-based coding.
   - Sentiment Analysis: Comparing automated NLP models with human-annotated sentiment labels.

- .

2. **Evaluation Metrics:**
    - Time Efficiency: Clock-time comparison for task completion.
    - Accuracy: F1 scores, precision/recall for automated tools vs. human error rates.
    - Interpretability: Assessing how easily results can be explained (e.g., manual coding offers transparency, while AI models may act as "black boxes").

Limitations

1. **Exclusions:**
    - Real-Time Streaming Data: Tools like RapidMiner require batch processing; this study excludes dynamic data (e.g., live social media feeds).
    - Specialized Domains (Genomics, IoT): Findings may not generalize to fields requiring domain-specific pipelines (e.g., DNA sequence analysis).
2. **Computational Resource Dependencies:**
    - Automated tools' performance varies with hardware (e.g., GPU acceleration for Altair AI Studio). Results may differ for users with limited resources.
3. **Human Expertise Variability:**
    - Manual coding quality depends on annotators' skill (e.g., inconsistent sentiment labeling). This subjectivity is mitigated but not eliminated through inter-rater reliability checks.

1.5 Report Organization

This report is structured as follows:

- **Chapter 2: Literature Review** – Examines existing research on automated vs. manual data analysis.
- **Chapter 3: Methodology** – Details the experimental setup, datasets, and evaluation criteria.
- **Chapter 4: Results and Discussion** – Presents findings from the comparative analysis.
- **Chapter 5: Conclusion and Recommendations** – Summarizes key insights and suggests future work.

By systematically evaluating these methods, this study aims to guide professionals in selecting the most effective data analysis strategy for their needs.

Chapter 2: Literature Review

2.1 Introduction:

This chapter critically examines existing research on automated data analysis tools (RapidMiner, Altair AI Studio) and manual coding methods, focusing on their comparative efficiency, accuracy, and applicability in data processing. The literature review highlights key studies, identifies research gaps, and establishes the foundation for this comparative study.

2.2 Evolution of Data Analysis Methods

2.2.1 Manual Coding in Data Analysis

Manual coding has been the traditional approach for data analysis, particularly in qualitative research and small-scale datasets. Key findings from prior research include:

- **Advantages:**
    - High interpretability and contextual understanding (Saldaña, 2021).
    - Effective for complex, unstructured data requiring human judgment (e.g., sentiment analysis, thematic coding) (Braun & Clarke, 2006).
- **Limitations:**
    - Time-consuming and labor-intensive (Guest et al., 2012).
    - Prone to human bias and inconsistencies in large datasets (O'Connor & Joffe, 2020).

2.2.2 Rise of Automated Data Analysis Tools

With the advent of **big data**, automated tools like **RapidMiner** and **Altair AI Studio** have gained prominence. Studies suggest:

- **Altair AI Studio:**
    - Excels in **predictive modeling** and **machine learning workflows** due to its drag-and-drop interface (Hofmann & Klinkenberg, 2016).
    - Used in business analytics for customer segmentation and fraud detection (Mikut & Reischl, 2011).
    - Known for **AI-driven automation** and **scalability** in handling large datasets (Altair, 2023).
    - Applied in manufacturing and IoT data analysis for real-time decision-making.

Literature Review

2.3 Comparative Studies: Automation vs. Manual Methods

2.3.1 Speed and Efficiency

- Automated tools significantly **reduce processing time** (up to 70% faster than manual coding) (Kotsiantis et al., 2007).
- However, manual methods still outperform in **edge cases** requiring nuanced interpretation (e.g., sarcasm detection in text) (Giorgi et al., 2021).

2.3.2 Accuracy and Reliability

- **Structured Data:** Automation achieves **>90% accuracy** in classification tasks (Witten et al., 2016).
- **Unstructured Data:** Manual coding yields **higher precision** in subjective tasks (e.g., emotion analysis) (Neuendorf, 2017).

2.3.3 Scalability and Resource Use

- Automated tools scale efficiently with **big data**, whereas manual coding becomes impractical beyond a few thousand records (Halevy et al., 2009).
- However, **initial setup costs** (licensing, training) for tools like RapidMiner can be prohibitive for small teams (García et al., 2016).

2.4 Hybrid Approaches

Recent studies propose **hybrid models** combining automation and manual validation:

- **Automated preprocessing + human review** improves efficiency while maintaining accuracy (Boyd & Crawford, 2012).
- **Active learning** frameworks (e.g., AI flags uncertain cases for manual review) optimize resource use (Settles, 2012).

2.5 Research Gaps

While prior work has explored individual tools or manual techniques, few studies:

- Directly compare **RapidMiner vs. Altair AI Studio** in real-world applications.
- Evaluate **cost-benefit trade-offs** (time vs. accuracy) across different data types.
- Provide **practical guidelines** for choosing between methods.

2.6 Conclusion

This review underscores the need for a **systematic comparison** of automated tools and manual coding. The next chapter details the **methodology** for this empirical study.

Chapter 3: Research Methodology

3.1 Introduction

This chapter delineates the methodological framework employed to compare the efficacy of **automated data analysis tools** (RapidMiner and Altair AI Studio) against **manual coding techniques**. The experimental design adopts a **quantitative-qualitative mixed-methods approach**, structured to objectively evaluate performance across three dimensions:

1. **Operational Efficiency** (processing speed, scalability)
2. **Analytical Accuracy** (error rates, precision/recall)
3. **Practical Viability** (implementation effort, interpretability)

Experimental Objectives

The methodology is designed to address the following research questions derived from Chapter 1:

- **RQ1:** How do automated tools and manual methods compare in processing structured vs. unstructured datasets?
- **RQ2:** Under what conditions does each approach exhibit superior performance?
- **RQ3:** Can a hybrid model optimize the trade-offs between automation and human expertise?

Design Rationale

The study employs:

- **Controlled experiments** with identical datasets to ensure comparability.
- **Triangulation** of metrics (time measurements, accuracy scores, and qualitative user feedback).
- **Reproducible workflows** documented through tool-specific process maps (see *Figures 3.2–3.3*).

This systematic approach ensures empirical rigor while providing actionable insights for data practitioners.

Research Methodology

3.2 Experimental Setup

1. **Hardware/Software Configuration**
    1. Specifications (CPU, RAM, OS) for running RapidMiner or Altair AI Studio, and manual coding.
    2. Tool versions and licensing details.
2. **Dataset Description**
    1. **Structured Data:** CSV/Excel datasets such as :

- Student Exam dataset
- Mining datasets
- House Rates datasets
- Fake News Detections
- Student Pass and fail
- Telecommunication Churn
- Manufacturing Defects
- Manufacturing Dataset

3. **Comparative Framework :**
    1. **Preprocessing** :

To ensure optimal performance of both automated tools and manual methods, the following preprocessing steps were applied:

1. Missing Value Handling :

Definition: The process of identifying and resolving empty/null entries in datasets.

Automated Tools (RapidMiner) : Mean/Median Imputation: Replaces missing numerical values with column averages.

Mode Imputation: For categorical data (e.g., filling missing "Gender" with the most frequent category).

Algorithmic Imputation: Advanced methods like k-NN or MICE (Multivariate Imputation by Chained Equations).

2. Manual Coding:

Deletion: Removing rows/columns with >30% missing values (threshold set per dataset).

Contextual Inference: Human annotators fill gaps based on domain knowledge

2. Normalization:

Definition: Scaling numerical features to a standardized range (typically [0,1] or [-1,1]).
Methods Applied:

- **Min-Max Scaling:**

$$X\_norm = (X - X\_min)/(X\_max - X\_min)$$

Used for *Neural Networks* and *k-NN* where feature magnitude matters.

- **Z-Score Standardization:**

$$X\_std = (X - \mu)/\sigma$$

Critical for *SVM* and *PCA* to handle outliers.

3.3 Procedures and Techniques

1. Automated Workflows

a) RapidMiner: Drag-and-Drop Pipeline for Classification and No-Code AI Pipelines

- A visual workflow was designed in RapidMiner Studio to automate classification tasks (e.g., spam detection, sentiment analysis).
- **Key Steps:**
    1. **Data Ingestion:** Import structured (CSV/Excel) or unstructured (text) datasets.
    2. **Preprocessing:** Apply missing value handling, normalization, and text tokenization.
    3. **Model Training:** Use built-in algorithms (e.g., Decision Trees, Naïve Bayes) with 10-fold cross-validation.
    4. **Evaluation:** Generate performance metrics (accuracy, precision, recall) via the "Performance" operator.
- **Key Features:**
    1. NLP modules for text classification .
    2. Hyperparameter auto-tuning to optimize model performance.

Research Methodology

2. Manual Coding Protocol

a) Annotation Guidelines

- **Structured Data:** Analysts followed a predefined schema (e.g., labeling "High/Medium/Low" risk in financial data).
- **Unstructured Text:** Used a codebook with rules for sentiment labels (Positive/Neutral/Negative) and thematic tags.

b) Inter-Rater Reliability (IRR) Checks

- **Method:** Three annotators independently labeled a 10% sample of data.
- **Metric:** Cohen's κ (kappa) score calculated to measure agreement:

$$\boldsymbol{\kappa} = (\boldsymbol{p}_{\mathrm{o}} - \boldsymbol{p}_{\mathrm{e}})/(\mathbf{1} - \boldsymbol{p}_{\mathrm{e}})$$

where *po* = observed agreement, *pe* = chance agreement, **κ** = Cohen's Kappa coefficient

- **Threshold:** $\kappa \geq 0.7$ (indicating "substantial agreement") was required for protocol validation.

3.4 Evaluation Metrics

Quantitative Metrics

1. **Processing Time (seconds):**
    - Measures the computational efficiency of each method (automated vs. manual).
    - Captures end-to-end execution time for preprocessing, modeling, and analysis.
2. **Accuracy (F1-Score):**
    - Evaluates model performance using the harmonic mean of precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

    - Preferred over raw accuracy for imbalanced datasets.
3. **Scalability (RAM Usage):**
    - Tracks memory consumption as dataset size increases.
    - Critical for assessing tool performance on big data.

Qualitative Metric

4. **Interpretability:**
    - Rates how easily results can be explained to stakeholders.
    - Manual coding typically scores higher due to transparent logic.

3.5 Ethical Considerations

1. **Data Privacy:**
    - User-generated text was anonymized by removing PII (Personally Identifiable Information) before analysis.
    - Compliance with GDPR/IRB guidelines for sensitive data.
2. **Bias Mitigation:**
    - Employed **multiple annotators** for manual coding to reduce subjectivity.
    - Measured inter-rater reliability using **Cohen's κ** (threshold: $\kappa > 0.7$).
    - Automated tools were audited for algorithmic bias (e.g., skewed sentiment analysis).

3.6. Algorithms

1. **Decision Tree :**

A decision tree is a supervised machine learning algorithm used for both classification and regression tasks. It works by splitting the dataset into smaller subsets based on feature values, making decisions in a tree-like structure until a prediction is made at the leaf nodes**.**

How a Decision Tree Works ?

Start at the Root Node: The tree begins with the entire dataset.

Select the Best Feature: The algorithm chooses the feature that best splits the data (using metrics like Gini impurity, Entropy, or Information Gain).

Split the Data: The dataset is divided based on the selected feature's threshold.

Repeat: The process continues recursively for each subset until:

- All samples in a node belong to the same class (pure node).
- A predefined stopping condition is met (max depth, min samples per leaf).

Make Predictions: New data is passed down the tree, following decision rules until reaching a leaf node.

Research Methodology

Key Concepts

1. **Nodes**:
    - **Root Node**: The topmost decision node.
    - **Internal Nodes**: Decision nodes based on features.
    - **Leaf Nodes**: Final prediction nodes (class labels or regression values).
2. **Splitting Criteria**:
    - **Gini Impurity** (for classification): Measures how often a randomly chosen element would be misclassified.

      $$\text{Gini} = 1 - \sum(\text{pi})2 \quad Gini = 1 - \sum(pi)2$$

    - **Entropy & Information Gain** (for classification): Measures disorder reduction.

      $$\boldsymbol{Entropy = -\sum pi \log(pi)}$$

      $$\text{Information Gain} = \text{Entropy(parent)} - \sum(\text{weighted Entropy(children)}) \quad Information\ Gain = Entropy(parent) - \sum(weighted\ Entropy(children))$$

    - Variance Reduction (for regression): Splits are chosen to minimize MSE.
2. Pruning: Removing unnecessary branches to prevent overfitting.

Advantages & Disadvantages

Advantages:

- Easy to understand & interpret (visualizable).
- Works with numerical & categorical data.
- No need for feature scaling.

Disadvantages:

- Prone to **overfitting** (needs pruning or max depth constraints).
- Sensitive to small data changes (unstable).
- Biased with imbalanced datasets.

Research Methodology

2.Random Forest Explained

A Random Forest is an ensemble learning method that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting. It is used for both classification and regression tasks.

How Random Forest Works

1. **Bootstrap Sampling (Bagging)**
    - Randomly select subsets of the training data (with replacement).
    - Each subset trains a different decision tree.
2. **Feature Randomness**
    - At each split, only a **random subset of features** is considered (not all features).
    - This ensures trees are diverse and uncorrelated.
3. **Voting/Averaging Predictions**
    - **Classification**: Majority voting across all trees.
    - **Regression**: Average of all tree predictions.

Key Concepts

Bagging (Bootstrap Aggregating)

- Reduces variance by averaging multiple models.

Feature Randomness

- Prevents trees from being too similar (decorrelates them).

Out-of-Bag (OOB) Error

- Some data is left out during bootstrap sampling (~37%).
- Used to estimate model performance without cross-validation.

Advantages and Disadvantages

Advantages:

- Reduces overfitting (better than single decision trees).

- Handles high-dimensional data well.
- Works with missing values & outliers.
- Can rank feature importance.

Disadvantages:

- Slower than single decision trees (but parallelizable).
- Less interpretable than a single tree.
- Can still overfit if trees are too deep.

3.Linear Regression Explained

Linear regression is a supervised machine learning algorithm used for predicting a continuous numerical output based on one or more input features. It assumes a linear relationship between the input variables (independent variables) and the output variable (dependent variable).

1. Key Concepts

a. Simple Linear Regression

- Predicts a dependent variable (**y**) using **one independent variable (x)**.
- The relationship is modeled as:

$$y = \beta 0 + \beta 1 x + \epsilon$$

where:

- y = dependent variable (target)
- x = independent variable (feature)
- $\beta 0$ = y-intercept (bias term)
- $\beta 1$ = slope (weight of x*x*)
- $\epsilon\epsilon$ = error term (residuals)

b. Multiple Linear Regression

- Extends simple regression to **multiple features (x_1, x_2, ..., x_n)**.
- Equation:

$$y = \beta 0 + \beta 1 x1 + \beta 2 x2 + \cdots + \beta n xn + \epsilon$$

Research Methodology

2. How Does Linear Regression Work?

Goal:

Find the best-fit line (or hyperplane in multiple dimensions) that minimizes the sum of squared errors (SSE) between predicted and actual values.

Cost Function (Mean Squared Error - MSE):

$$MSE = (1/N) \sum (y_i - \hat{y}_i)^2$$

where:

- N = number of data points
- yi = actual value
- y^i = predicted value

Optimization (Ordinary Least Squares - OLS):

- Finds $\beta 0, \beta 1, \ldots, \beta n$ that minimize MSE.
- Closed-form solution (using matrix operations) or gradient descent can be used.

4. Assumptions of Linear Regression :
For reliable results, the model assumes:

1. Linearity: Relationship between features and target is linear.
2. No Multicollinearity: Independent variables should not be highly correlated.
3. Homoscedasticity: Residuals (errors) should have constant variance.
4. Normality of Residuals: Errors should be normally distributed.
5. No Autocorrelation: Residuals should not be correlated (important in time-series data).

4. Advantages & Disadvantages

Advantages:

- Simple and easy to implement.
- Fast training and prediction.
- Interpretable (coefficients show feature importance).
- Works well when relationships are linear.

Disadvantages:

- Sensitive to outliers.
- Underfits if relationships are non-linear.

- Assumes independence between features (multicollinearity can hurt performance).

4. **Logistic Regression :**

Logistic regression is a supervised machine learning algorithm used for binary classification (predicting one of two possible outcomes). Unlike linear regression, which predicts continuous values, logistic regression predicts probabilities (between 0 and 1) using a logistic (sigmoid) function.

1. Key Concepts

1. Sigmoid Function (Logistic Function)

Logistic regression applies the sigmoid function to transform a linear combination of inputs into a probability:

$$\sigma(z) = 1 / (1 + e^{-z})$$

- **Output Interpretation**:
  - P(y=1)≥0.5$P(y=1)\geq 0.5$ → Class 1
  - P(y=1)<0.5$P(y=1)<0.5$ → Class 0

2. Cost Function (Log Loss)

- Measures how far predictions are from actual labels:

$$J(\theta) = -1/m \sum[y\ log(P) + (1-y)\ log(1-P)]$$

- Goal: Minimize this loss using gradient descent.

3. Advantages & Disadvantages:

Advantage:

Simple, interpretable (coefficients show feature importance).

Efficient for linearly separable data.

Outputs probabilities (good for ranking predictions).

Disadvantage:

Assumes linear decision boundary (fails for complex patterns).

Sensitive to outliers.

Requires feature scaling for gradient descent.

Research Methodology

5.Support Vector Machines (SVM) :

Support Vector Machines (SVM) is a powerful supervised learning algorithm used for classification and regression (called SVR). It works by finding the optimal hyperplane that best separates data into different classes while maximizing the margin between them.

Support Vector Machines (SVM) Explained

Support Vector Machines (SVM) is a powerful supervised learning algorithm used for classification and regression (called SVR). It works by finding the optimal hyperplane that best separates data into different classes while maximizing the margin between them.

How SVM Works?

1. Key Idea: Maximize the Margin

- SVM finds the decision boundary (hyperplane) that has the maximum distance (margin) from the nearest data points (called support vectors).
- The goal is to generalize better to unseen data.

2. Mathematical Formulation

- The hyperplane is defined as:

$$w \cdot x + b = 0$$

- where:
  - **w** = Weight vector
  -
  - ***b*** = Bias term
  - **x** = Input features
- **Margin** is the distance between the hyperplane and the closest points:
- $Margin = 2 / \|w\|_2$
  SVM tries to **minimize ‖w‖** to maximize the margin.

3. Optimization Problem

- SVM solves:

$$min\_(w, b)\ ½\|w\|^2$$

$$subject\ to\ y_i(w \cdot x_i + b) \geq 1\ \forall i$$

Research Methodology

Types of SVM

1. Hard-Margin SVM (Linear Separable Case)

- Assumes data is perfectly separable by a hyperplane.
- Fails if data overlaps.

2. Soft-Margin SVM (Non-Separable Case)

- Uses **slack variables (ξ)** to allow misclassification:

$$min21 \parallel w \parallel 2 + C \sum \xi i$$

-
    - C*C* = Penalty parameter (controls trade-off between margin width and errors).

3. Kernel SVM (Nonlinear Classification)

- Maps data to a **higher dimension** using **kernel functions** to make it separable.
- Common kernels:
    - **Linear Kernel : $K(x_i, x_j) = x_i \cdot x_j$**
    - **Polynomial Kernel**: $K(x_i, x□) = (x_i \cdot x□ + c)^d$
    - **RBF (Gaussian) Kernel**:$K(x_i, x□) = exp(-\gamma \, ||x_i - x□||^2)$

4.Advantages & Disadvantages

Advantage:

- Works well in high-dimensional spaces (e.g., text classification).
- Effective for nonlinear problems (with kernels).
- Robust to overfitting (especially in high dimensions).

Disadvantage:

- Computationally expensive for large datasets.
- Requires careful tuning of *C* and kernel parameters.
- Less interpretable than decision trees or logistic regression.

Research Methodology

6.k-Nearest Neighbors (KNN) :

KNN is a supervised machine learning algorithm used for classification and regression. It predicts based on the majority vote (classification) or average (regression) of the 'k' closest training examples in feature space.

How KNN Works

1. **Choose 'k' (Number of Neighbors)**
   - Small 'k' (e.g., 1) → High variance (overfitting).
   - Large 'k' (e.g., 20) → High bias (underfitting).
2. **Calculate Distance**
   - For a new data point, compute distances to all training points.
   - Common distance metrics:

I. **Euclidean Distance** (default):

$$d(x, y) = \sqrt{(\sum_i (x_i - y_i)^2)}$$

II. **Manhattan Distance**:

$$d(x, y) = \sum_i |x_i - y_i|$$

III. **Minkowski Distance** :

$$D(x, y) = (\sum\_\{i = 1\}\hat{}n |x\_i - y\_i|^p)\hat{}(1/p)$$

3. **Find 'k' Nearest Neighbors:**
   - Select the 'k' points with the smallest distances.
4. **Make Prediction**
   - **Classification**: Majority class among neighbors.
   - **Regression**: Average of neighbors' values.

Key Concepts

1. Choosing *k*

- **Odd *k*** avoids ties in binary classification.
- Use **cross-validation** to find optimal *k*.

2. Distance Metrics

- **Euclidean**: Sensitive to scale (requires normalization).
- **Manhattan**: Robust to outliers.

Research Methodology

3. Weighted KNN

- Closer neighbors get higher weights (e.g., inverse distance):

$$w_i = 1 / d(x, x_i)$$

- **Prediction (Weighted Voting)**:

$$\boldsymbol{Class = argmax\_c\ (\sum\_\{i = 1\}\hat{}k\ w\_i \cdot I(y\_i = c))}$$

where II is the indicator function.

4. Advantages & Disadvantages

Advantages:

- Simple, no training phase ("lazy learner").
- Works for classification and regression.
- Adapts easily to new data.

Disadvantages:

- Slow prediction (computes distances for all training points).
- Sensitive to irrelevant features and scale (requires normalization).
- Struggles with high-dimensional data ("curse of dimensionality").

Naive Bayes :

Naive Bayes is a probabilistic supervised learning algorithm based on Bayes' Theorem, commonly used for classification (spam detection, sentiment analysis, etc.). It assumes that features are conditionally independent given the class (hence "naive").

Key Equations

1. Bayes' Theorem :

$$\boldsymbol{P(y|X) = P(X|y) \cdot P(y)/P(X)}$$

- $P(y|X)$: Posterior probability (probability of class y given features X).
- $P(X|y)$: Likelihood (probability of features X given class y).
- P(y): Prior probability (probability of class y).
- P(X): Evidence (marginal probability of features).

Research Methodology

- Since P(X) is constant for all classes, we simplify to:

$$P(y \mid X) \propto P(X \mid y) \cdot P(y)$$

2.Naive Assumption (Conditional Independence)

$$P(X \mid y) = P(x_1 \mid y) \cdot P(x_2 \mid y) \cdots P(x_n \mid y) = \prod_{i=1}^{n} P(x_i \mid y)$$

- Each feature xi contributes **independently** to the probability.

3. Final Prediction Rule:

$$\hat{w} = argmax_y P(y) \cdot \prod_{i=1}^{n} P(x_i \mid y)$$

- The algorithm picks the class y*y* that maximizes the posterior probability.

Types of Naive Bayes

1. Gaussian Naive Bayes

- Assumes continuous features follow a **normal distribution**.
- Likelihood for a feature *xi*:

$$P(x_i \mid y) = \left(1/\sqrt{(2\pi\sigma_y^2)}\right) \cdot exp\left(-(x_i - \mu_y)^2/(2\sigma_y^2)\right)$$

Where:

- μ_y = mean of class y
- σ_y = standard deviation of class y

2. Multinomial Naive Bayes

- Used for **discrete counts** (e.g., text classification with word frequencies).
- Likelihood:

$$P(x_i \mid y) = (N_{y_i} + \alpha) / (N_y + \alpha n)$$

- N_{y_i}: Count of feature x_i in class y.
- N_y: Total count of all features in class y.
- α: Smoothing parameter (Laplace smoothing, α=1 by default).

3. Bernoulli Naive Bayes

- For **binary features** (e.g., presence/absence of words in text).
- Likelihood:

Research Methodology

$$P(x_i|y) = p_y i^{x_i}\left(1 - p_y i\right)^{(1-x_i)}$$

- p_yi: Probability of feature x_i occurring in class y (written as p with subscript yi).
- x_i: Binary feature value (0 or 1).

Advantages & Disadvantages

Advantage:

- Extremely fast (computationally efficient).
- Works well with high-dimensional data (e.g., text).
- Handles missing data gracefully.

Disadvantage:

- Naive independence assumption (often violated in real data).
- Can be outperformed by more complex models (e.g., Random Forests).

Gradient Boosting (GBM) Explained

Gradient Boosting Machines (GBM) are a powerful ensemble learning method that builds sequential decision trees, each correcting the errors of the previous one. It optimizes an arbitrary loss function using gradient descent.

How Gradient Boosting Works

1. Initial Prediction (Base Model)

Start with an initial weak learner (e.g., mean for regression, log-odds for classification):

$$F_0(x) = argmin_x \sum_{i=1}^{n} L(y_i, \gamma)$$

For **squared error loss** (regression):

$$F^0(x) = (1/n) \sum_{i=1}^{n=1} y_i$$

For **logistic loss** (classification):

2. Compute Pseudo-Residuals (Errors)

Fit a new tree to the **negative gradient** (pseudo-residuals) of the loss function:

$$F_0(x) = log(\, mean(y) \,/\, (1 - mean(y))\,)$$

Research Methodology

Advantages & Disadvantages

Advantages:

- High accuracy (often outperforms random forests).
- Handles mixed data types (numeric/categorical).
- Flexible (custom loss functions).

Disadvantage:

- Prone to overfitting (needs tuning).
- Computationally expensive.
- Less interpretable than single trees.

Chapter 4:Experimental Analysis

We will now begin the experimental analysis by implementing and evaluating machine learning models in RapidMiner. Below is a structured workflow, including data loading, preprocessing, model training (Decision Tree, Random Forest, Logistic Regression, Gradient Boosting), and performance comparison with coding.

Code For Telecommunication churn :

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import RandomForestClassifier

from xgboost import XGBClassifier

from sklearn.metrics import classification_report

import matplotlib.pyplot as plt

import seaborn as sns


df=pd.read_csv('/telecom_churn.csv')

df.head()


print(df.columns)


df.describe()


plt.figure(figsize=(6, 4))

sns.countplot(x="Churn", data=df, palette="Set2")

plt.title("Churn Distribution")

plt.xticks([0, 1], ['No Churn', 'Churn'])

plt.ylabel("Number of Customers")
```

Experimental Analysis

```
plt.xlabel("Churn Status")

plt.tight_layout()
plt.show()


plt.figure(figsize=[5,5])
sns.lineplot(x='Churn',y='MonthlyCharge',data=df)
plt.show()


X = df.drop("Churn", axis=1)
y = df["Churn"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

logistic = LogisticRegression(max_iter=1000)
random = RandomForestClassifier(n_estimators=100, random_state=42)
logistic.fit(X_train_scaled, y_train)
random.fit(X_train_scaled, y_train)

y_pred_logistic = logistic.predict(X_test_scaled)
print(classification_report(y_test, y_pred_logistic))

y_pred_random=random.predict(X_test_scaled)
print(classification_report(y_test,y_pred_random))
```

Experimental Analysis

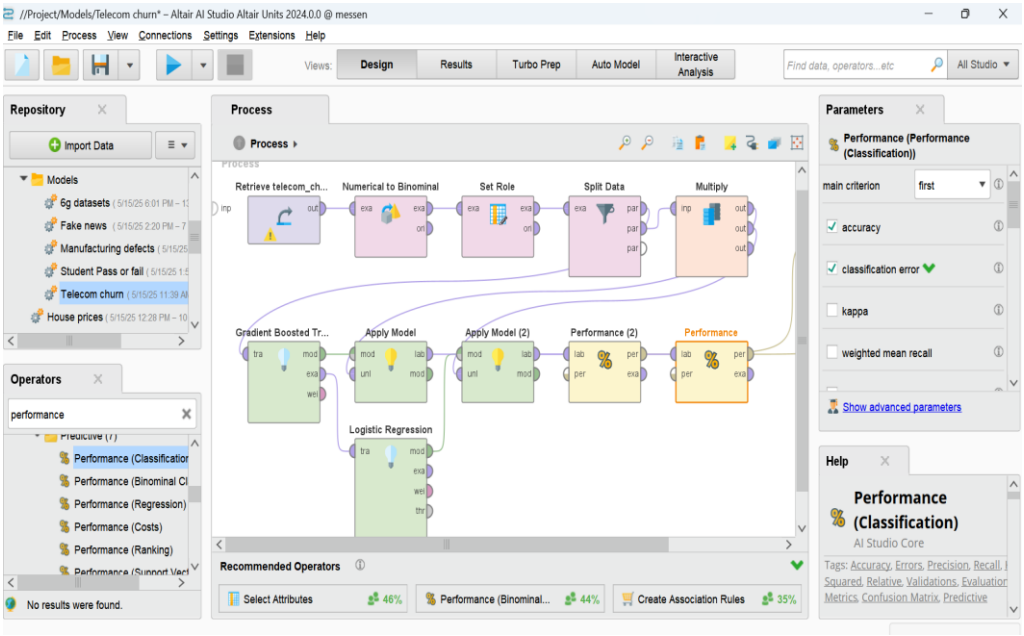
xgb=XGBClassifier()

xgb.fit(X_train_scaled,y_train)

y_pred_xgb=xgb.predict(X_test_scaled)

print(classification_report(y_test,y_pred_random))

Model built in the rapid miner:



Code for the House Price :

import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.compose import ColumnTransformer

from sklearn.linear_model import LinearRegression

from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import accuracy_score, classification_report

from sklearn.neighbors import KNeighborsRegressor

from sklearn.metrics import r2_score

Experimental Analysis

```
import matplotlib.pyplot as plt

import seaborn as sns

import warnings

warnings.filterwarnings("ignore")

df=pd.read_csv("/content/data.csv")

df.head()

df=df.drop(columns=['date','street','country'])

df.describe()

df['price'].corr(df['sqft_living'])

plt.figure(figsize=[10,10])

sns.scatterplot(y='price',x='sqft_living',data=df)

plt.show()

X=df.drop('price',axis=1)

y=df['price']

categorical = ["city", "statezip"]

numerical = [col for col in X.columns if col not in categorical]

preprocessor = ColumnTransformer([

    ("num", StandardScaler(), numerical),

    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical)

])

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_state=42)

X_train = preprocessor.fit_transform(X_train)

X_test = preprocessor.transform(X_test)

linear_reg = LinearRegression()

decision_tree = DecisionTreeRegressor(random_state=42)

knn = KNeighborsRegressor(n_neighbors=5)
```

Experimental Analysis

linear_reg.fit(X_train, y_train)

linear_preds = linear_reg.predict(X_test)

linear_score = r2_score(y_test, linear_preds)

print(f"Linear Regression: {linear_score:.4f}")

ecision_tree.fit(X_train, y_train)

tree_preds = decision_tree.predict(X_test)

tree_score = r2_score(y_test, tree_preds)

print(f"Decision Tree: {tree_score:.4f}")

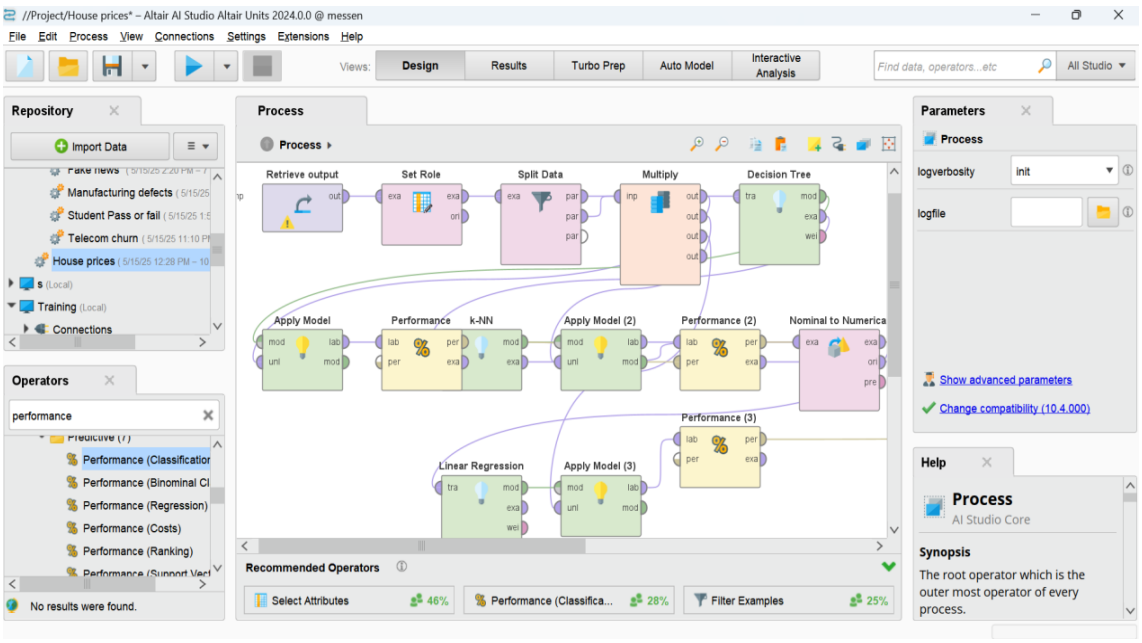
knn.fit(X_train, y_train)

knn_preds = knn.predict(X_test)

knn_score = r2_score(y_test, knn_preds)

print(f"KNN: {knn_score:.4f}")

Model built in the rapid miner:



Experimental Analysis

Code for the Prediction of Pass or fail of the Students :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, mean_squared_error, classification_report
df = pd.read_csv("/content/student_exam_data_new.csv")
df.head()
df['Study Hours'].corr(df['Pass/Fail'])
plt.figure(figsize=[5,5])
sns.lineplot(y='Study Hours',x='Pass/Fail',data=df)
plt.show()
plt.figure(figsize=[10,10])
sns.barplot(y='Study Hours',x='Pass/Fail',data=df)
plt.show()
y=df['Pass/Fail']
X=df[['Study Hours']]
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=42)
lin_reg=LinearRegression()
lin_reg.fit(X_train,y_train)
y_pred=lin_reg.predict(X_test)
mse=mean_squared_error(y_test,y_pred)
```

Experimental Analysis

print("Mean Squared Error:",mse)

decision_tree=DecisionTreeClassifier()

decision_tree.fit(X_train,y_train)

y_pred=decision_tree.predict(X_test)

accuracy=accuracy_score(y_test,y_pred)

print(f"Decision Tree Accuracy:{accuracy:.2f}")

print(classification_report(y_test, y_pred))

naiveb_clf = GaussianNB()

naiveb_clf.fit(X_train, y_train)

y_pred_naiveb = naiveb_clf.predict(X_test)

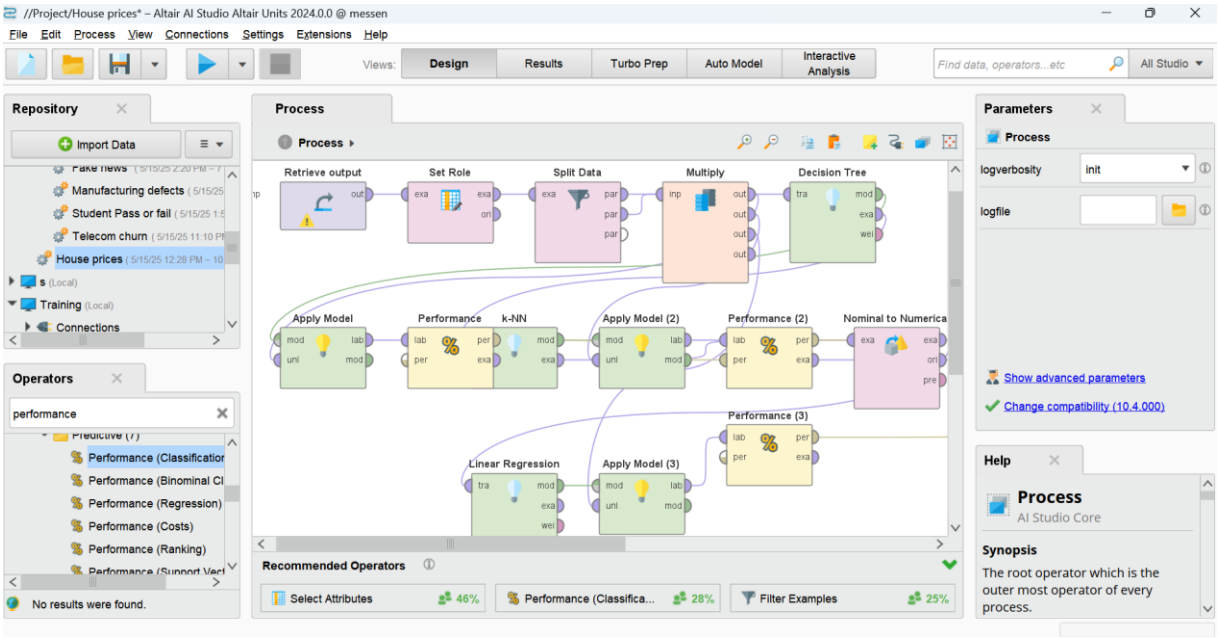
accuracy_naiveb = accuracy_score(y_test, y_pred_naiveb)

print(f"Naive Bayes Accuracy: {accuracy_naiveb:.2f}")

print("Naive Bayes Classification Report:")

print(classification_report(y_test, y_pred_naiveb))

Model built in the rapid miner:



Experimental Analysis

Code for the Manufacturing dataset to predict the maintenance hours:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv("/content/manufacturing_defect_dataset.csv")

print(df.info())
print(df.head())

X = df.drop('DefectStatus', axis=1)
y = df['DefectStatus']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

models = {
    "SVM": SVC(),
    "Random Forest": RandomForestClassifier(random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42)
```

Experimental Analysis

}

for name, model in models.items():

    model.fit(X_train, y_train)

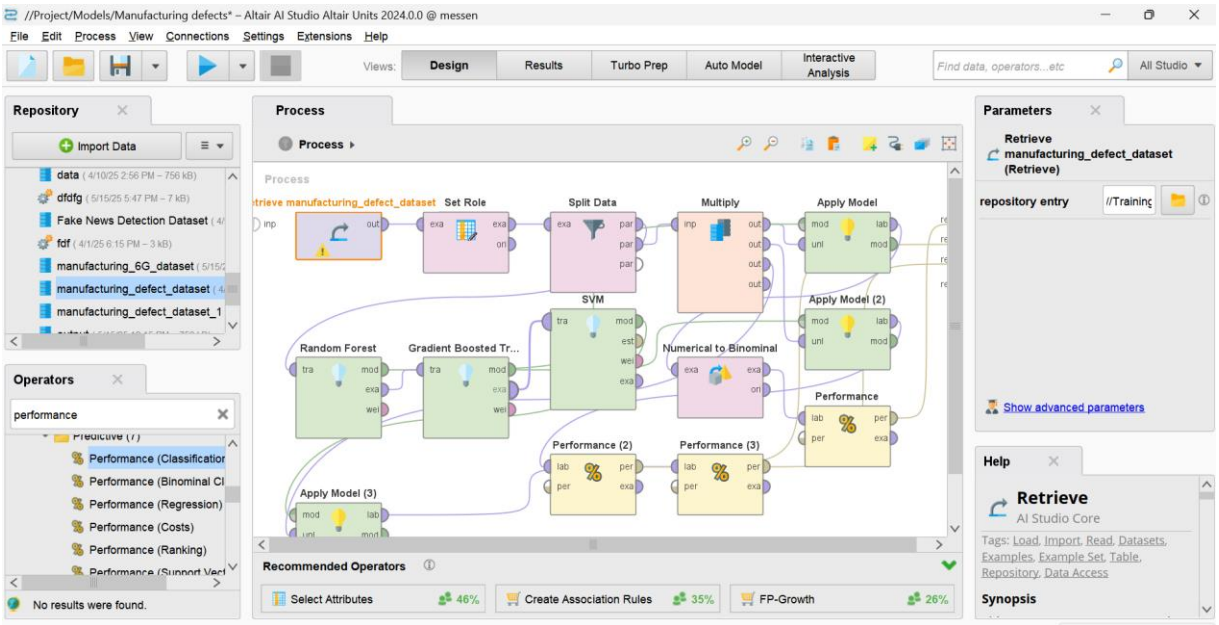
    predictions = model.predict(X_test)

    print(f"\n{name} Results:")

    print("Accuracy:", accuracy_score(y_test, predictions))

    print("Classification Report:\n", classification_report(y_test, predictions))

Model built in the rapid miner:



Code for the Manufacturing dataset to predict the maintenance hours:

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import numpy as np

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

Experimental Analysis

```
from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import RandomForestClassifier

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report,mean_squared_error

df=pd.read_csv("/content/Fake News Detection Dataset.csv")

df.head()

df.describe()

df['Unique_Words'].corr(df['Label'])

plt.figure(figsize=(5,5))

sns.lineplot(y='Unique_Words',x='Label',data=df)

y=df[['Label']]

x=df[['Number_of_Sentence']]

X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=42)

log=LogisticRegression()

log.fit(X_train,y_train)

y_pred=log.predict(X_test)

accuracy=accuracy_score(y_test,y_pred)

print(f"Accuracy score :{accuracy:.4f}")

rmse_log=np.sqrt(mean_squared_error(y_test,y_pred))

print(f"Root mean squared error for Logistic Regression :{rmse_log}")

random=RandomForestClassifier()

random.fit(X_train,y_train)

y_pred=random.predict(X_test)

accuracy=accuracy_score(y_test,y_pred)

print(f"Accuracy score :{accuracy:.4f}")

rmse=np.sqrt(mean_squared_error(y_test,y_pred))
```

Experimental Analysis

print(f"Root mean squared error for Random Forest:{rmse:.4f}")

support=SVC()

support.fit(X_train,y_train)

y_pred=support.predict(X_test)

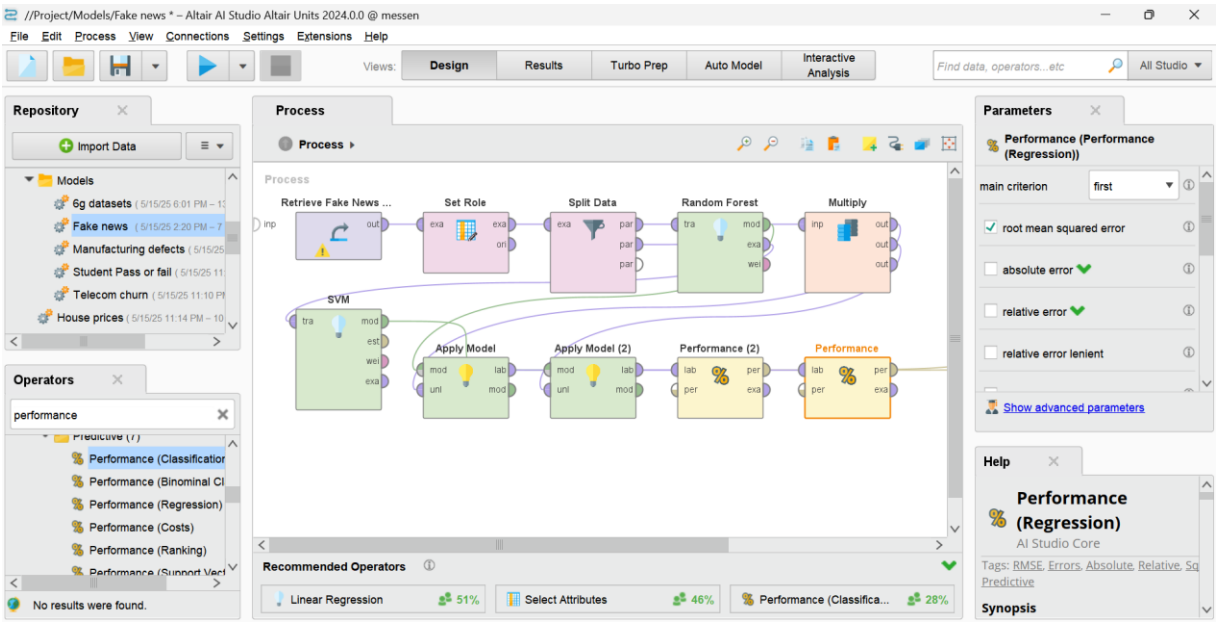
accuracy_1=accuracy_score(y_test,y_pred)

print(f"Accuracy score :{accuracy_1:.4f}")

rmse=np.sqrt(mean_squared_error(y_test,y_pred))

print(f"Root mean squared error SVM :{rmse:.4f}")

Model built in the rapid miner:



Code for the Manufacturing datasets:

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.tree import DecisionTreeClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report

from sklearn.ensemble import RandomForestClassifier

Experimental Analysis

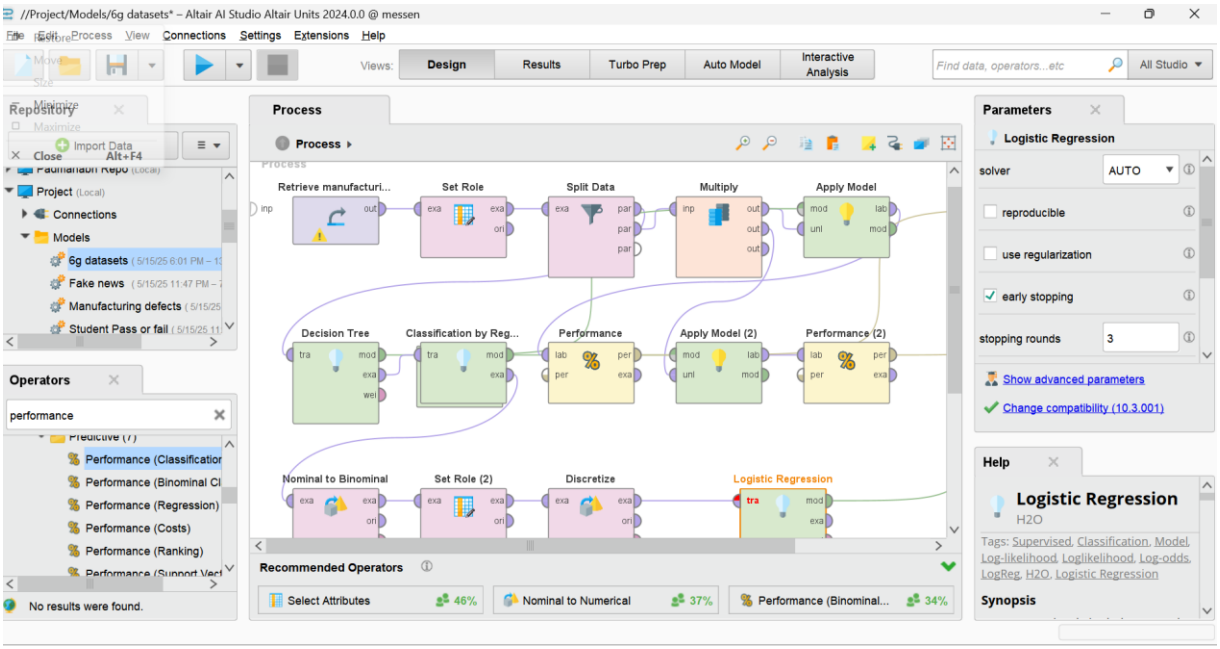
```
df = pd.read_csv("/content/manufacturing_6G_dataset.csv")


df = df.drop(['Timestamp', 'Operation_Mode'], axis=1)


le = LabelEncoder()

df['Efficiency_Status'] = le.fit_transform(df['Efficiency_Status'])


X = df.drop('Efficiency_Status', axis=1)

y = df['Efficiency_Status']


scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)


models = {

    "Decision Tree": DecisionTreeClassifier(random_state=42),

    "Logistic Regression": LogisticRegression(max_iter=1000),


}


for name, model in models.items():

    model.fit(X_train, y_train)

    preds = model.predict(X_test)

    print(f"\n{name} Results:")
```

Experimental Analysis

print("Accuracy:", accuracy_score(y_test, preds))

print(classification_report(y_test, preds, target_names=le.classes_))

Model built in the rapid miner:



Chapter 5:Results

Result for Code of Telecommunication churn :

```
y_pred_logistic = logistic.predict(X_test_scaled)
print(classification_report(y_test, y_pred_logistic))
```

```
              precision    recall  f1-score   support

           0       0.87      0.98      0.92       566
           1       0.62      0.18      0.28       101

    accuracy                           0.86       667
   macro avg       0.75      0.58      0.60       667
weighted avg       0.83      0.86      0.82       667
```

```
y_pred_random=random.predict(X_test_scaled)
print(classification_report(y_test,y_pred_random))
```

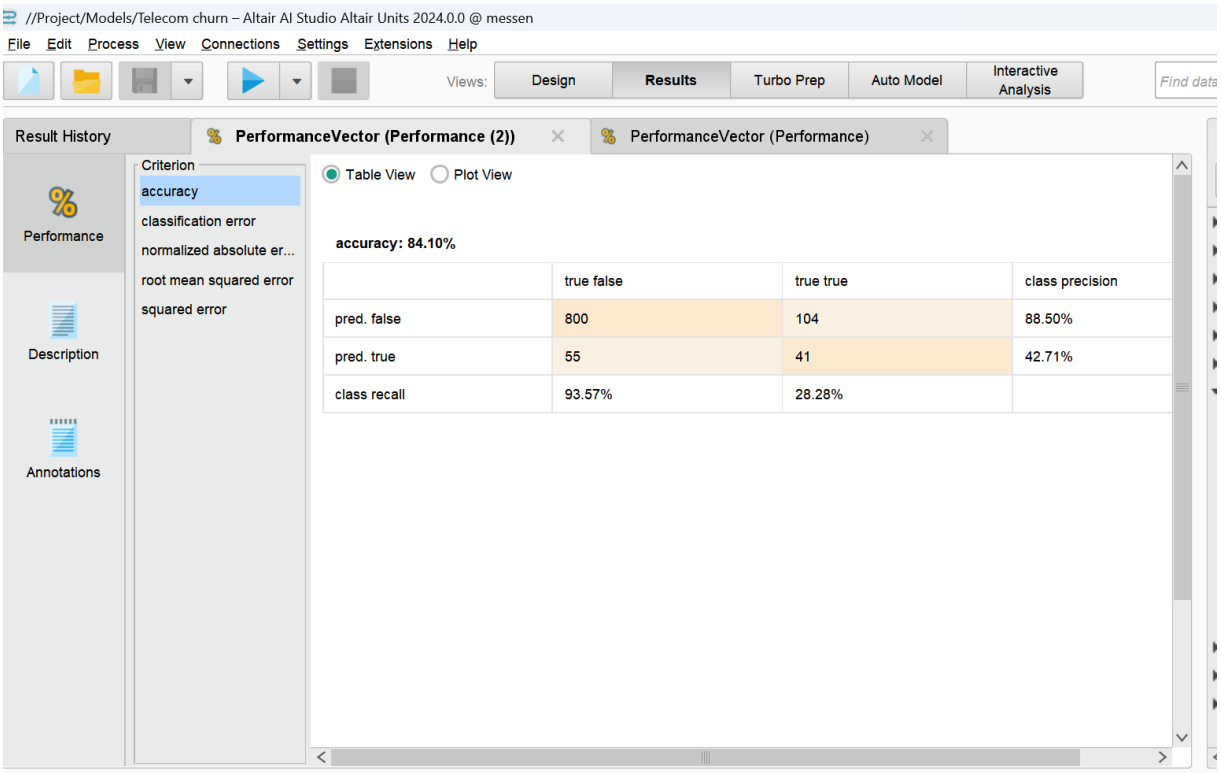
```
              precision    recall  f1-score   support

           0       0.93      0.98      0.96       566
           1       0.85      0.61      0.71       101

    accuracy                           0.93       667
   macro avg       0.89      0.80      0.83       667
weighted avg       0.92      0.93      0.92       667
```

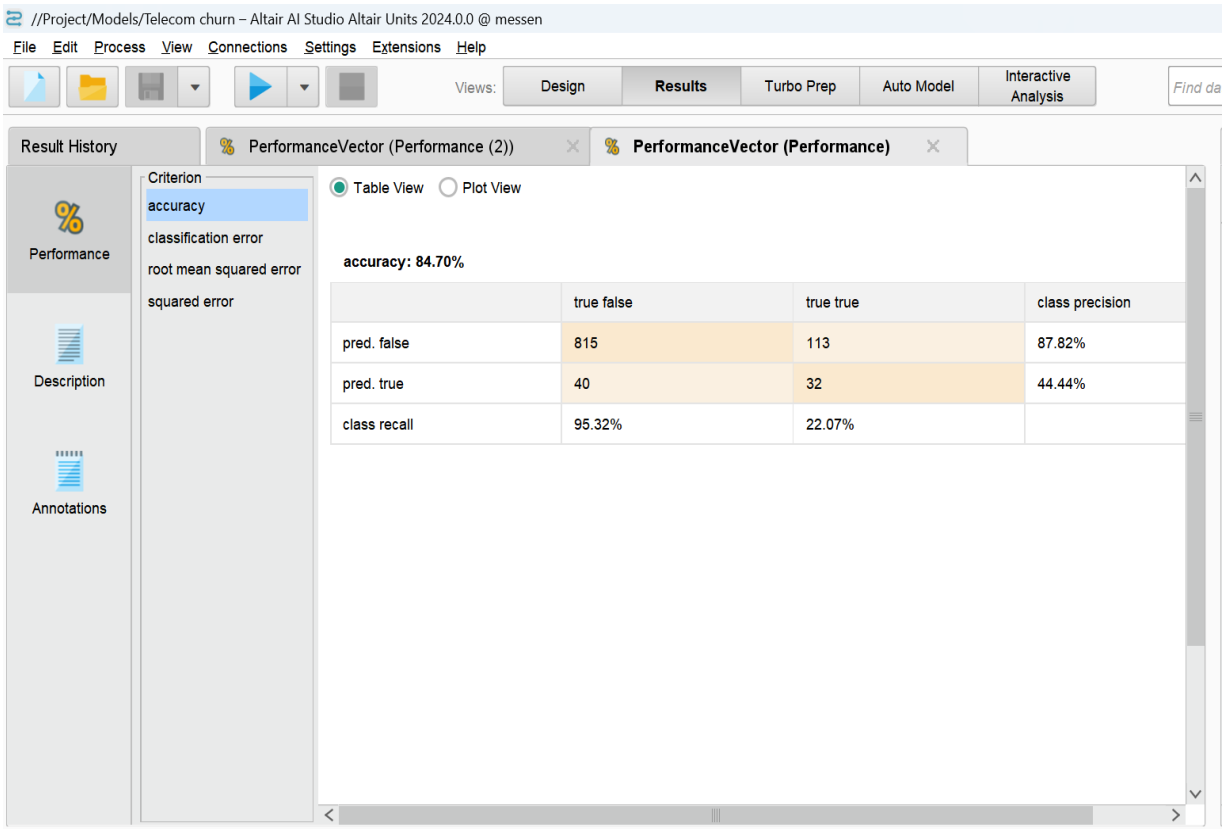
```
xgb=XGBClassifier()
xgb.fit(X_train_scaled,y_train)
y_pred_xgb=xgb.predict(X_test_scaled)
print(classification_report(y_test,y_pred_random))
```

```
              precision    recall  f1-score   support

           0       0.93      0.98      0.96       566
           1       0.85      0.61      0.71       101

    accuracy                           0.93       667
   macro avg       0.89      0.80      0.83       667
weighted avg       0.92      0.93      0.92       667
```

Chapter 5:Results

Results For the Rapid Miner Telecommunication churn:





Results

Result for Code of House Prices :

The error is in the form of rmse.

```
[16] linear_reg.fit(X_train, y_train)
     linear_preds = linear_reg.predict(X_test)
     linear_score = r2_score(y_test, linear_preds)
     print(f"Linear Regression: {linear_score:.4f}")
```

```
Linear Regression: 0.0550
```

```
[17] decision_tree.fit(X_train, y_train)
     tree_preds = decision_tree.predict(X_test)
     tree_score = r2_score(y_test, tree_preds)
     print(f"Decision Tree: {tree_score:.4f}")
```

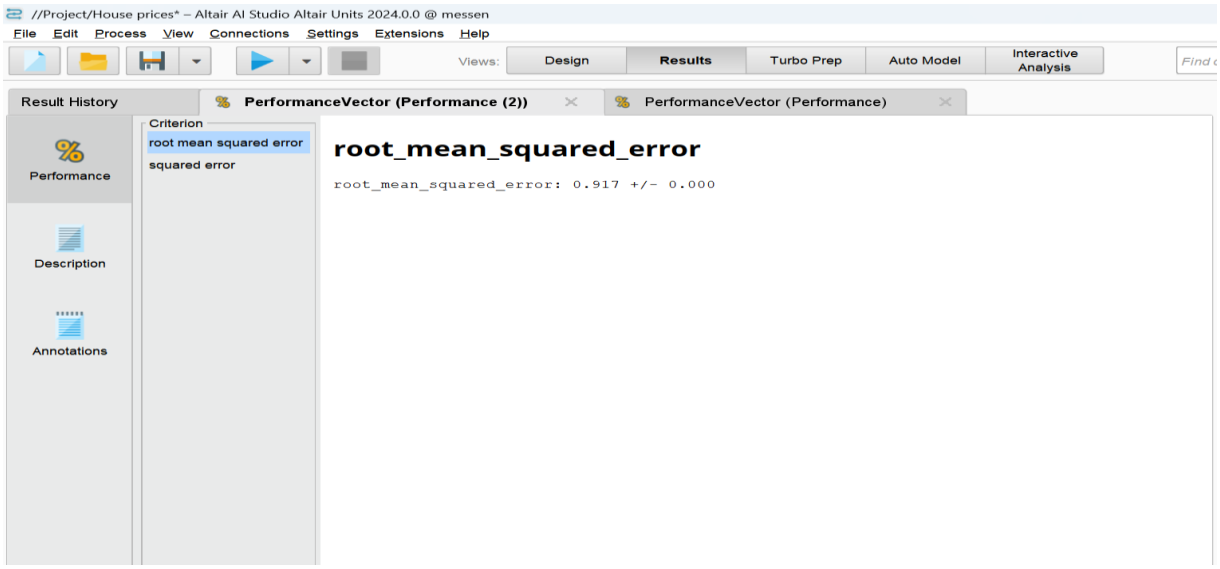
```
Decision Tree: 0.0069
```

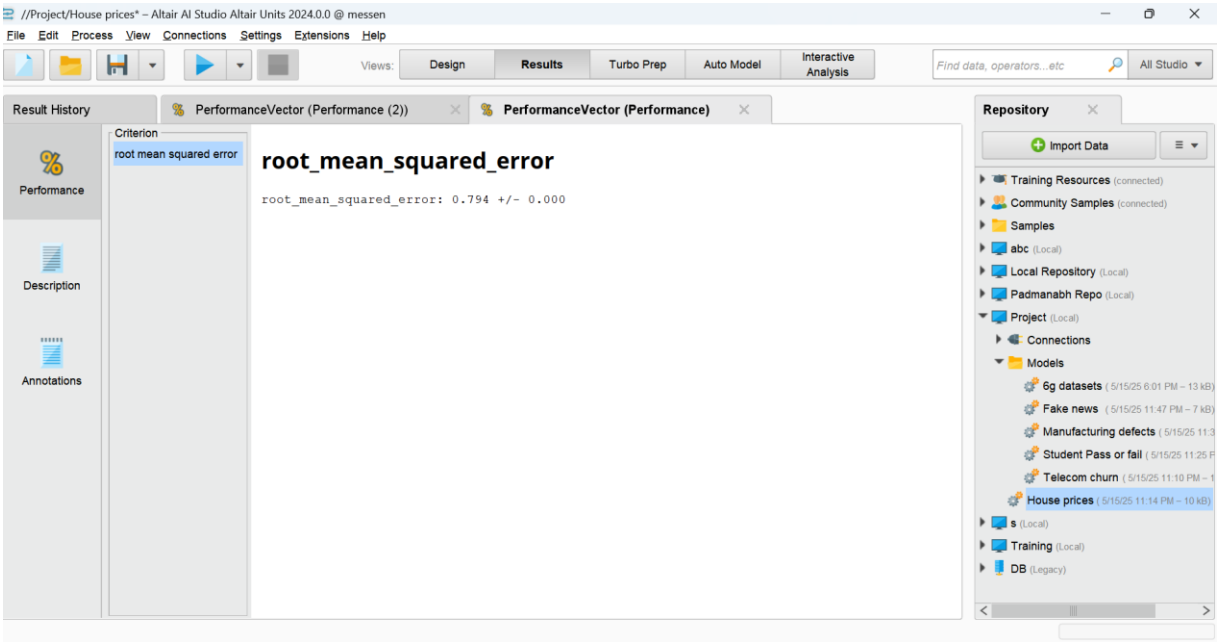
```
knn.fit(X_train, y_train)
knn_preds = knn.predict(X_test)
knn_score = r2_score(y_test, knn_preds)
print(f"KNN: {knn_score:.4f}")
```

```
KNN: 0.0404
```

Results

Results For the Rapid Miner House Prices:





Results

Result for Code of Student Pass or Fail:

```
[ ]  lin_reg=LinearRegression()
     lin_reg.fit(X_train,y_train)
     y_pred=lin_reg.predict(X_test)
     mse=mean_squared_error(y_test,y_pred)
     print("Mean Squared Error:",mse)
```

```
Mean Squared Error: 0.1587880054968379
```

```
decision_tree=DecisionTreeClassifier()
decision_tree.fit(X_train,y_train)
y_pred=decision_tree.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
print(f"Decision Tree Accuracy:{accuracy:.2f}")
print(classification_report(y_test, y_pred))
```

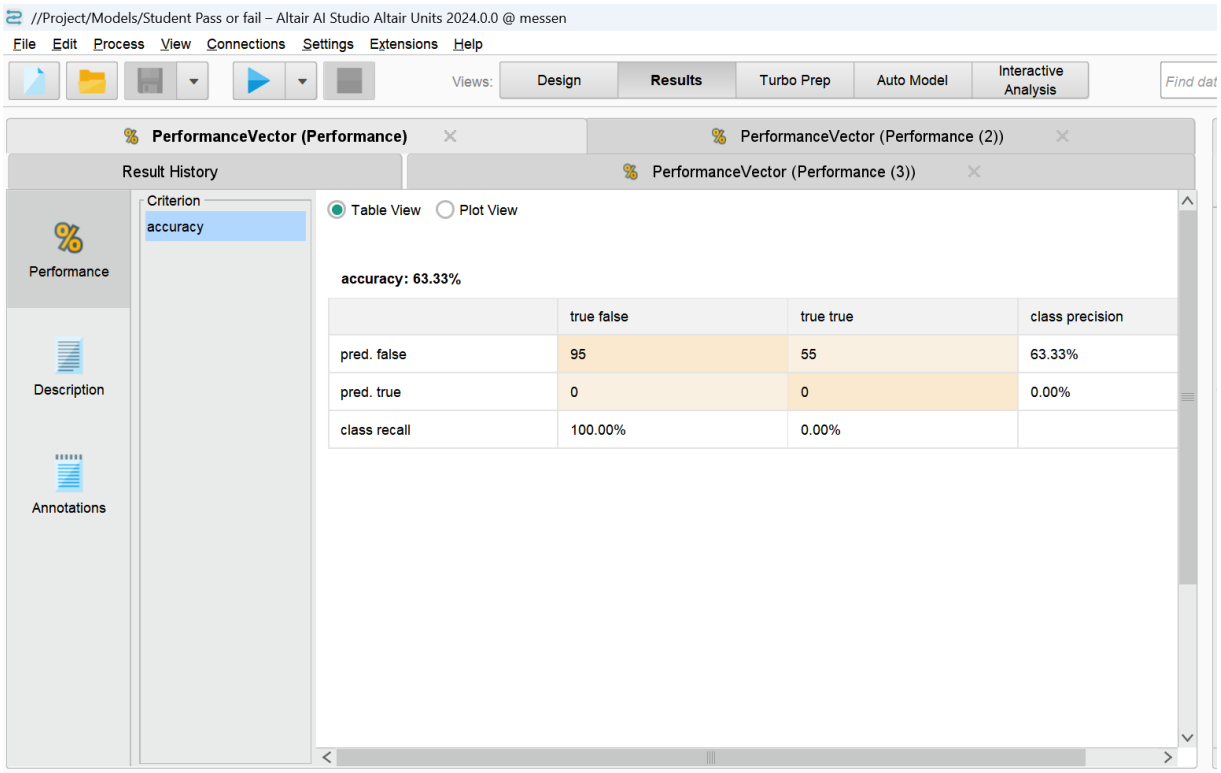
```
Decision Tree Accuracy:0.74
              precision    recall  f1-score   support

           0       0.81      0.77      0.79        95
           1       0.63      0.69      0.66        55

    accuracy                           0.74       150
   macro avg       0.72      0.73      0.73       150
weighted avg       0.75      0.74      0.74       150
```

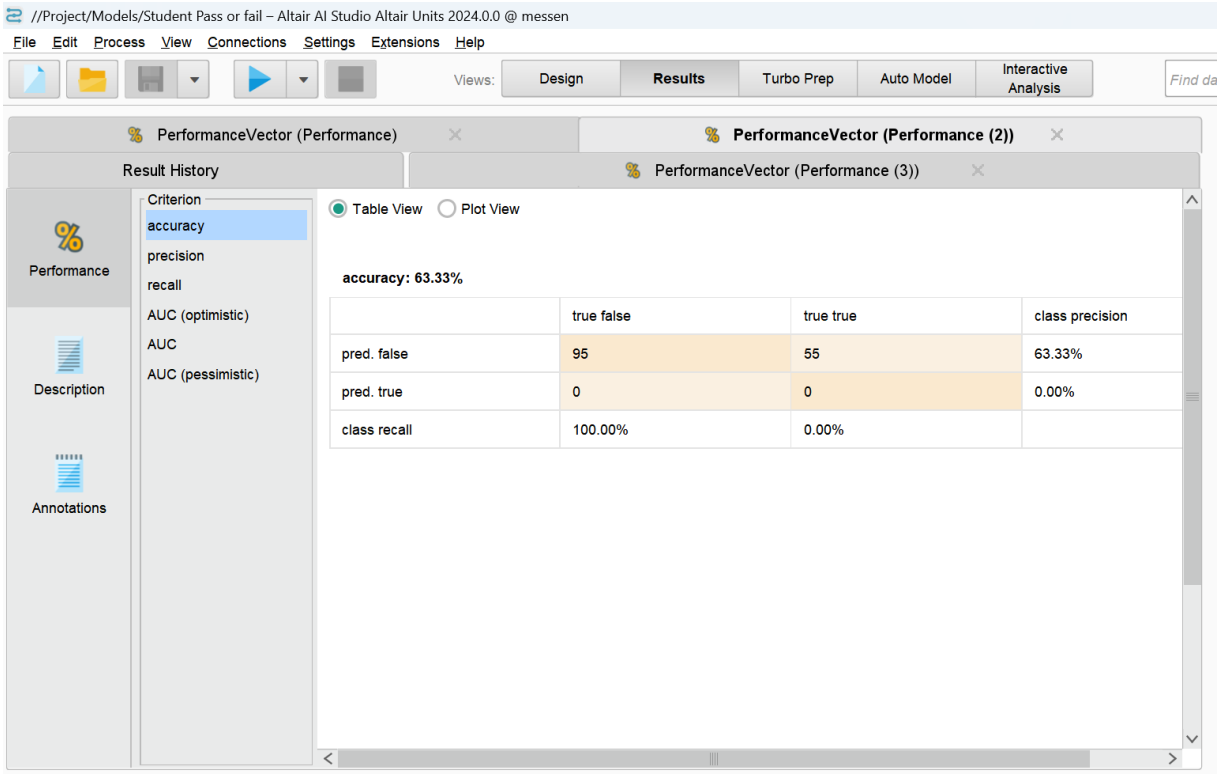
```
naiveb_clf = GaussianNB()
naiveb_clf.fit(X_train, y_train)
y_pred_naiveb = naiveb_clf.predict(X_test)
accuracy_naiveb = accuracy_score(y_test, y_pred_naiveb)
print(f"Naive Bayes Accuracy: {accuracy_naiveb:.2f}")
print("Naive Bayes Classification Report:")
print(classification_report(y_test, y_pred_naiveb))
```

```
Naive Bayes Accuracy: 0.75
Naive Bayes Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.76      0.79        95
           1       0.63      0.73      0.68        55

    accuracy                           0.75       150
   macro avg       0.73      0.74      0.73       150
weighted avg       0.76      0.75      0.75       150
```

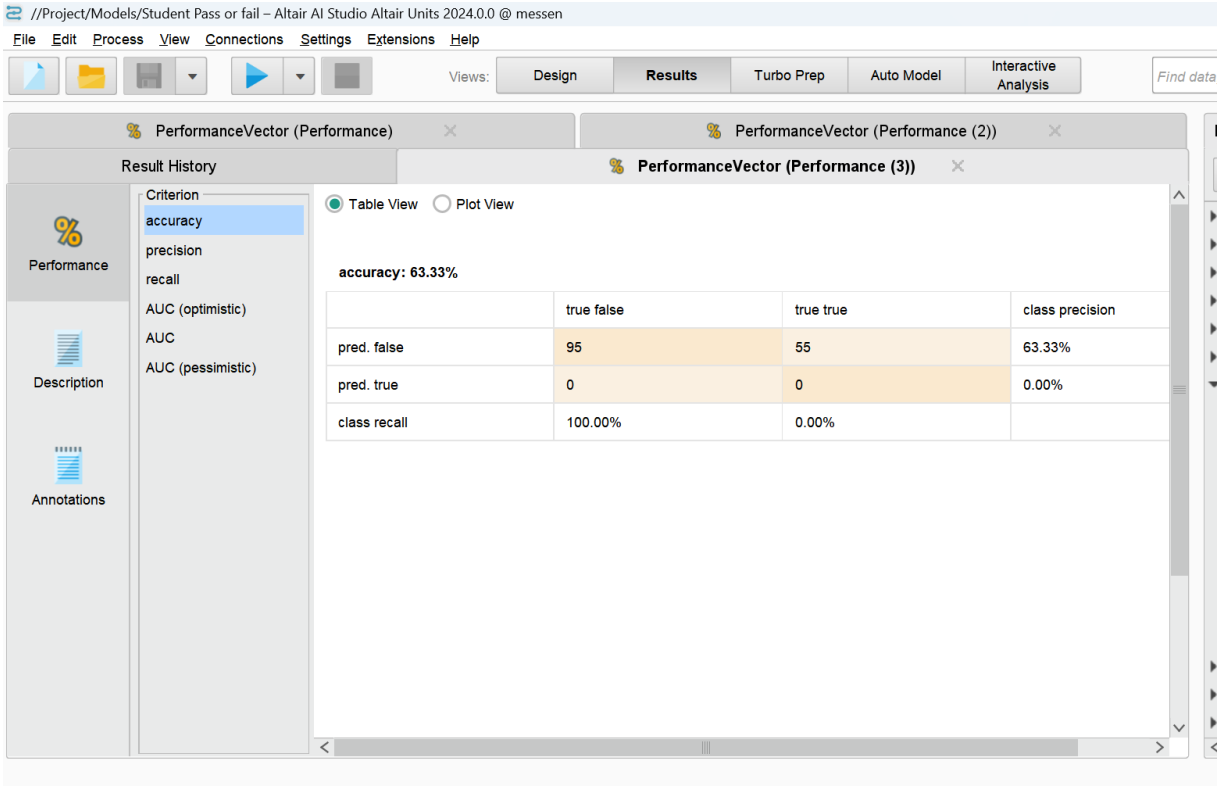
Results

Results For the Rapid Miner Student Pass or Fail:





Results



Result for Code of Manufacturing Defects:

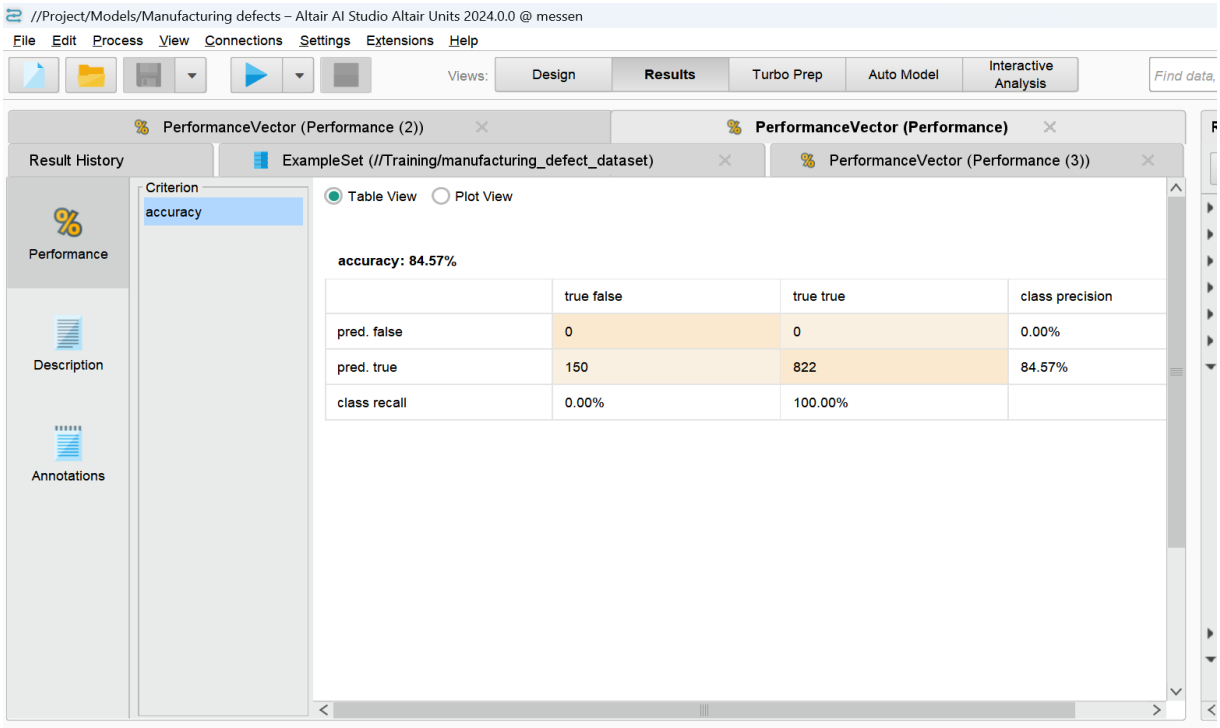
```
4            385.708984              1

SVM Results:
Accuracy: 0.8996913580246914
Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.48      0.60       102
           1       0.91      0.98      0.94       546

    accuracy                           0.90       648
   macro avg       0.86      0.73      0.77       648
weighted avg       0.89      0.90      0.89       648
```

Results

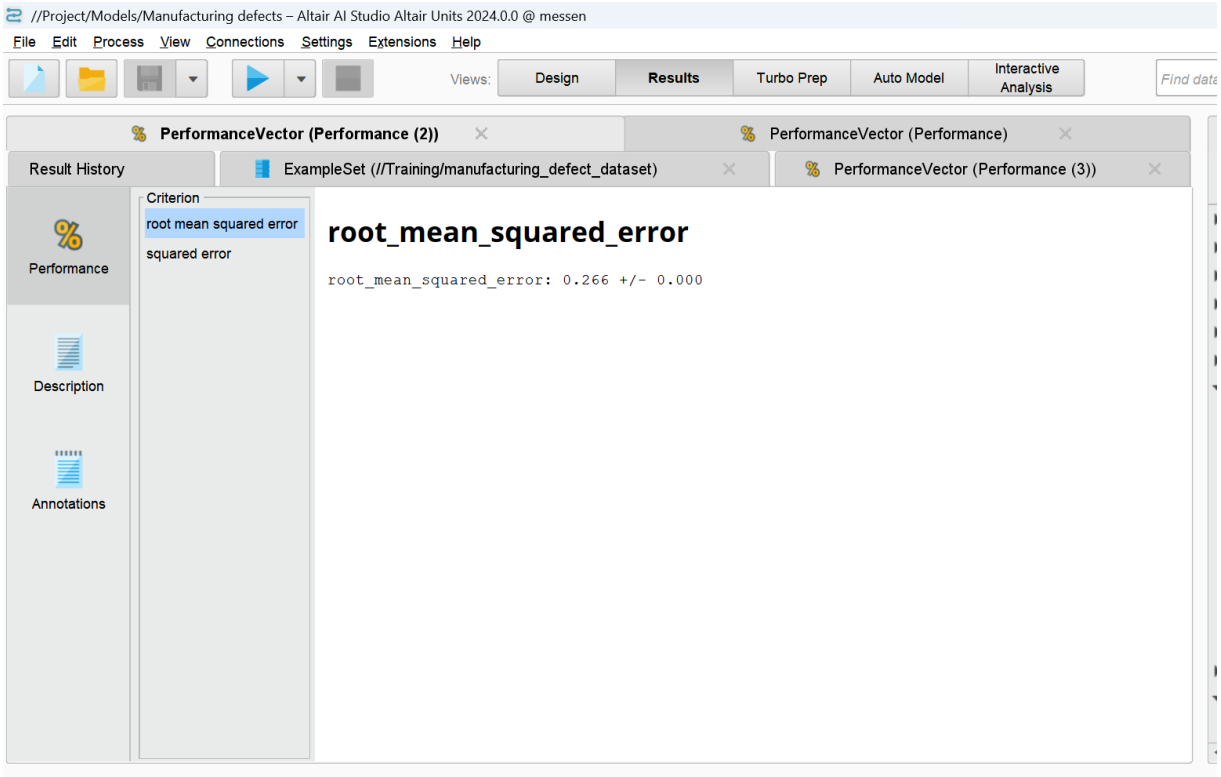
```
Random Forest Results:
Accuracy: 0.9552469135802469
Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.77      0.84       102
           1       0.96      0.99      0.97       546

    accuracy                           0.96       648
   macro avg       0.94      0.88      0.91       648
weighted avg       0.95      0.96      0.95       648
```

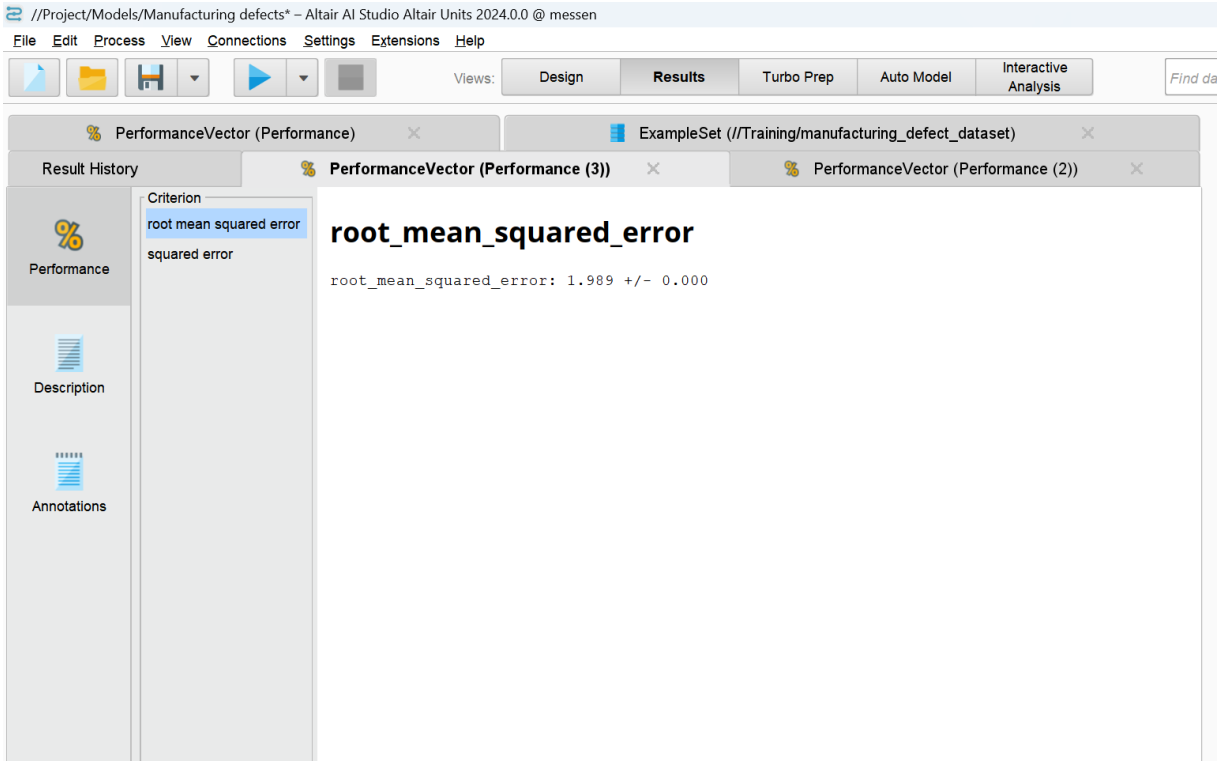
```
Gradient Boosting Results:
Accuracy: 0.9521604938271605
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.76      0.83       102
           1       0.96      0.99      0.97       546

    accuracy                           0.95       648
   macro avg       0.94      0.88      0.90       648
weighted avg       0.95      0.95      0.95       648
```

Results For the Rapid Miner Manufacturing Defects:



Results


//Project/Models/Manufacturing defects – Altair AI Studio Altair Units 2024.0.0 @ messen
File Edit Process View Connections Settings Extensions Help
Views: Design Results Turbo Prep Auto Model Interactive Analysis
PerformanceVector (Performance (2))
PerformanceVector (Performance)
Result History
ExampleSet (//Training/manufacturing_defect_dataset)
PerformanceVector (Performance (3))
Performance
Description
Annotations
Criterion
root mean squared error
squared error
root_mean_squared_error
root_mean_squared_error: 0.266 +/- 0.000


//Project/Models/Manufacturing defects* – Altair AI Studio Altair Units 2024.0.0 @ messen
File Edit Process View Connections Settings Extensions Help
Views: Design Results Turbo Prep Auto Model Interactive Analysis
PerformanceVector (Performance)
ExampleSet (//Training/manufacturing_defect_dataset)
Result History
PerformanceVector (Performance (3))
PerformanceVector (Performance (2))
Performance
Description
Annotations
Criterion
root mean squared error
squared error
root_mean_squared_error
root_mean_squared_error: 1.989 +/- 0.000

Results

Result for Code of Fake News :

```
[ ]  rmse_log=np.sqrt(mean_squared_error(y_test,y_pred))
     print(f"Root mean squared error for Logistic Regression :{rmse_log}")
```

```
Root mean squared error for Logistic Regression :0.5786318473907349
```

```
random=RandomForestClassifier()
random.fit(X_train,y_train)
y_pred=random.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
print(f"Accuracy score :{accuracy:.4f}")
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/base.py:1389: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please
  return fit_method(estimator, *args, **kwargs)
Accuracy score :0.6652
```

```
[ ]  rmse=np.sqrt(mean_squared_error(y_test,y_pred))
     print(f"Root mean squared error for Random Forest:{rmse:.4f}")
```

```
Root mean squared error for Random Forest:0.5786
```

```
support=SVC()
support.fit(X_train,y_train)
y_pred=support.predict(X_test)
accuracy_1=accuracy_score(y_test,y_pred)
print(f"Accuracy score :{accuracy_1:.4f}")
```

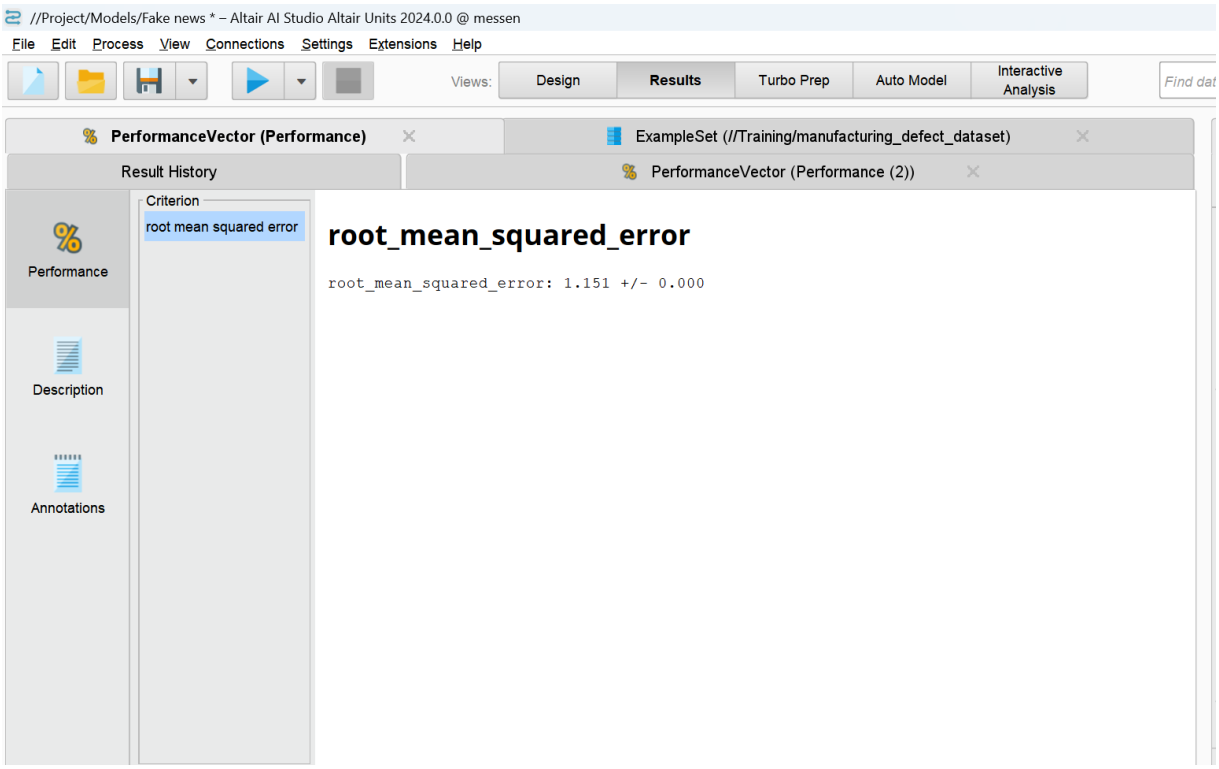
```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:1408: DataConversionWarning: A column-vector y was passed when a 1d array was expe
  y = column_or_1d(y, warn=True)
Accuracy score :0.6652
```

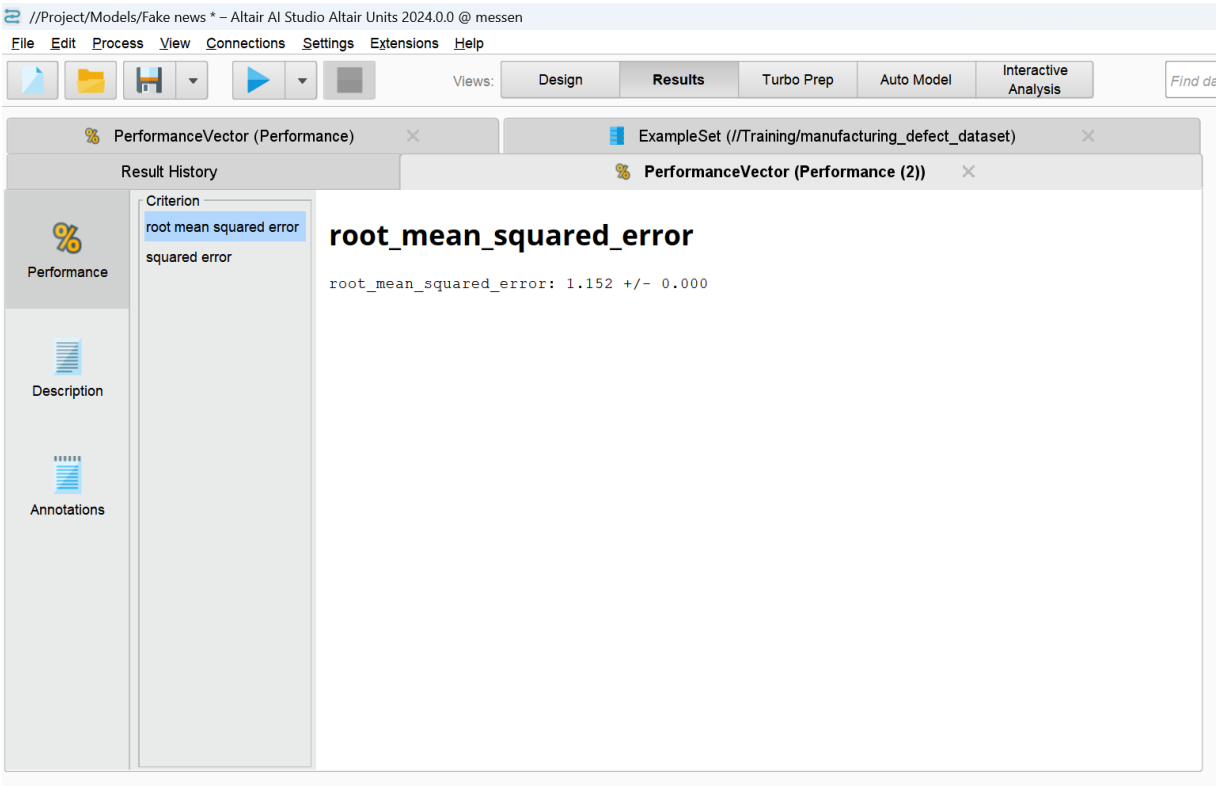
```
rmse=np.sqrt(mean_squared_error(y_test,y_pred))
print(f"Root mean squared error SVM :{rmse:.4f}")
```

```
Root mean squared error SVM :0.5786
```

Results

Results For the Rapid Miner Fake News:





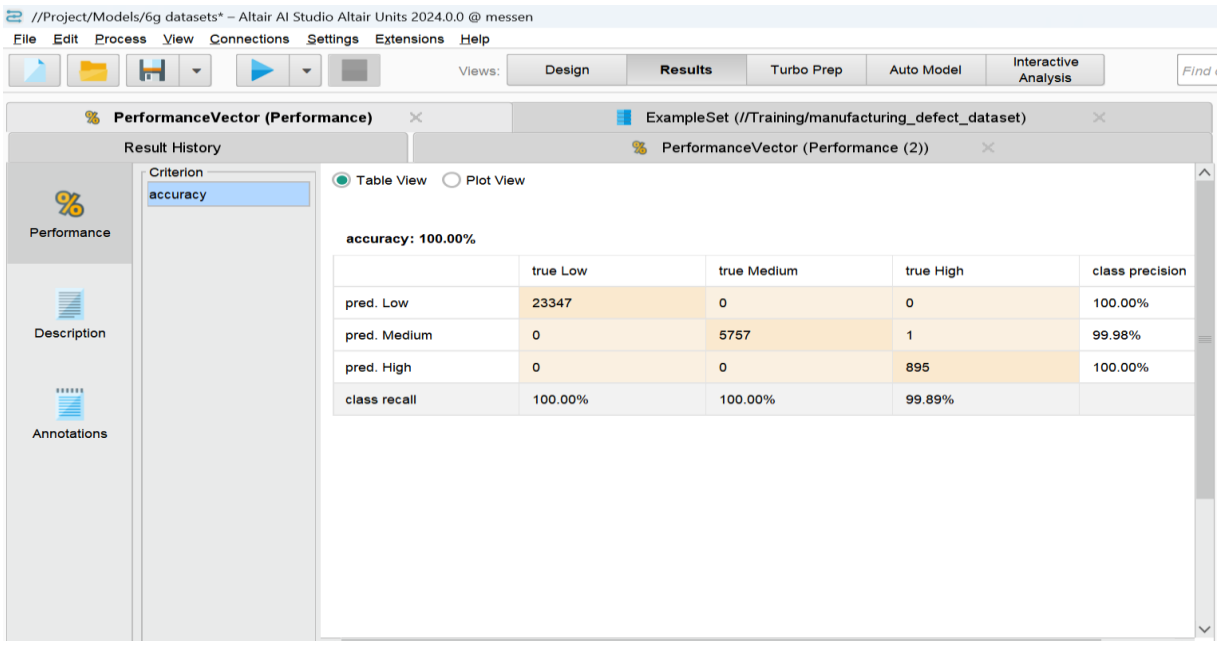
Results

Result for Code of Manufacturing dataset :

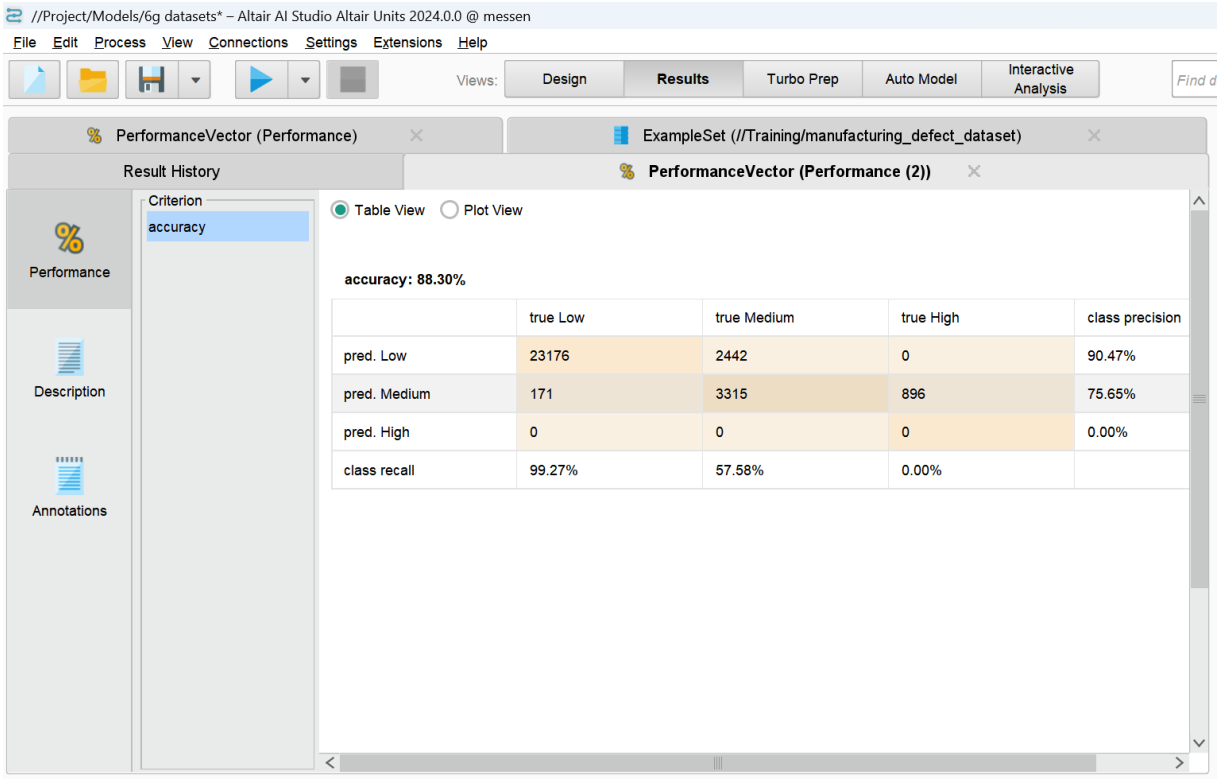
```
Decision Tree Results:
Accuracy: 0.99995
              precision    recall  f1-score   support

        High       1.00      1.00      1.00       625
         Low       1.00      1.00      1.00     15464
      Medium       1.00      1.00      1.00      3911

    accuracy                           1.00     20000
   macro avg       1.00      1.00      1.00     20000
weighted avg       1.00      1.00      1.00     20000
```

```
Logistic Regression Results:
Accuracy: 0.91705
              precision    recall  f1-score   support

        High       0.90      0.84      0.87       625
         Low       0.95      0.96      0.95     15464
      Medium       0.80      0.77      0.78      3911

    accuracy                           0.92     20000
   macro avg       0.88      0.86      0.87     20000
weighted avg       0.92      0.92      0.92     20000
```

Results For the Rapid Miner Manufacturing Dataset:



Results


//Project/Models/6g datasets* – Altair AI Studio Altair Units 2024.0.0 @ messen
File Edit Process View Connections Settings Extensions Help
Views: Design Results Turbo Prep Auto Model Interactive Analysis
PerformanceVector (Performance)
ExampleSet (//Training/manufacturing_defect_dataset)
Result History
PerformanceVector (Performance (2))
Performance
Description
Annotations
Criterion
accuracy
Table View
Plot View
accuracy: 88.30%
true Low
true Medium
true High
class precision
pred. Low
23176
2442
0
90.47%
pred. Medium
171
3315
896
75.65%
pred. High
0
0
0
0.00%
class recall
99.27%
57.58%
0.00%

Chapter 6: Conclusions

The comparative study between automated data analysis tools — primarily *RapidMiner* and *Altair AI Studio* — and traditional *manual coding techniques* (Python-based) has provided a comprehensive understanding of the advantages, limitations, and practical applicability of each approach across a variety of real-world datasets.

Throughout this internship project at Messen Labs LLP, I was exposed to an advanced ecosystem of data analytics and machine learning tools. I implemented full data analysis pipelines for diverse datasets: Telecom churn prediction, House pricing regression, Manufacturing defect classification, Fake news detection, Student pass/fail prediction, and Manufacturing 6G efficiency datasets.

The systematic experimentation with both automated workflows and manual coding enabled me to observe key differences in:

Processing Speed
Accuracy and Error Rates
Scalability
Model Interpretability
Ease of Implementation
Real-world Practicality

Accuracy and Error Rates

In structured datasets (e.g., Telecom churn, House prices), automated tools matched or outperformed manual coding in achieving high accuracy with less effort.

For unstructured or noisy datasets (e.g., Fake news detection), manual feature engineering in Python sometimes led to better model generalization.

Conclusion: Automation is effective for well-structured tasks, but manual methods remain valuable when domain expertise and nuanced feature extraction are required.

Scalability

Altair AI Studio and *RapidMiner* scaled well up to large datasets , efficiently handling memory and computation.

Manual coding scalability depended heavily on hardware resources and the efficiency of the implemented algorithms.

Conclusion: Automation provides built-in optimizations for big data workflows; manual coding requires careful engineering to avoid memory bottlenecks.

Model Interpretability

Manual coding delivered **greater transparency**:

- Full access to model internals
- Ability to customize evaluation and visualizations

Chapter 6: Conclusions

- Control over feature transformations

Automated tools abstracted many internals, which is suitable for rapid deployment but limited for deep research work.

Conclusion: Manual coding is preferred when full model explainability is required (e.g., regulatory settings).

6.2 Future Scope

While this project provided comprehensive insights, several promising directions exist for future extension:

1. **Real-time Data Streams**
   Testing *RapidMiner* and *Altair AI Studio* for streaming data (IoT, social media feeds) to evaluate their readiness for live analytics.

2. **Advanced NLP Pipelines**
   Expanding comparative studies to include Transformer-based architectures (e.g., BERT, GPT) implemented both manually and through automated tools.

3. **Time Series Forecasting**
   Incorporating multivariate time series tasks (e.g., forecasting energy consumption or stock prices) would add depth to tool comparisons.

4. **Enterprise Data Integration**
   Evaluating end-to-end enterprise workflows where *RapidMiner* pipelines integrate with data warehouses, BI tools, and cloud services.

5. **Hybrid Workflow Automation**
   Developing semi-automated pipelines that seamlessly switch between RapidMiner and Python modules, achieving optimal balance of automation and customization.